

Final Design Review

MSc Robotics AERO62520 Robotic Systems Design Project

Year of submission

2026

Team Number

Team 10

School of Engineering

Contents

Contents	2
1 Introduction	4
1.1 Purpose of the Document	4
1.2 Problem Statement	4
1.3 Project Objectives	4
1.4 Responses to Feedback from Preliminary Design Review	5
1.5 Summary of Modifications Since Preliminary Design Review	6
2 Sustainability Checklist	8
3 Cyber Security Considerations	10
4 System	11
4.1 System Architecture	11
4.2 Component Selection	12
5 Mechanical Design	16
5.1 3D SolidWorks Model	16
5.2 Component Placement Challenges	20
5.3 Stability & Manufacturability	22
5.3.1 General Considerations	22
5.3.2 Manipulator Platform Structural Analysis	22
5.3.3 Robot Stability and Centre of Mass Analysis	22
6 Electrical Design	25
6.1 Power Connection	25
6.2 Power Budget	26
7 Software Design	28
7.1 Software Architecture	28
7.2 Mission Execution Flowchart	32
7.3 ROS2 Interfaces	34
8 Analysis	35
8.1 Move and Navigation	35
8.2 Structure and Reliability	37
8.3 Power Source and Finish	39

8.4 Safety and Compliance	40
9 Project Plan	41
A Updated Design Requirements Document	44
A.1 Problem Statement	44
A.2 Concept of Operations (CONOPS)	44
A.3 Functional and Performance Requirements	46
A.4 Requirements Verification Matrix	48
B Updated Workplace Charter	52
B.1 Statement of Purpose	52
B.2 Statement of Principles and Commitments	52
B.3 Team Rules	52
B.4 Daily Activity Conflict Avoidance	53
B.5 Conflict Resolution Flowchart	55

Word count: 9322

1 Introduction

This section introduces the Final Design Review document, outlining the problem statement and objectives of the proposed robot design.

1.1 Purpose of the Document

The purpose of this Final Design Review is to present the initial design architecture of the robotic system, and to demonstrate that the proposed solution meets the functional and performance requirements defined in the previous ‘Design Requirements Analysis’. This document details the system-level, mechanical, electrical, and software designs of the mobile manipulator.

1.2 Problem Statement

Modern autonomous systems are defined by the robust integration of multiple complex subsystems. While individual skills like perception, navigation, and manipulation are often taught in isolation, real world industrial use cases often require unifying them to form a single cohesive and capable robotic platform.

This project serves as a scaled-down, canonical representation of this industrial problem. It demands mastery of challenging sub-problems including SLAM, computer vision, navigation, manipulation, and, crucially, their integration into a single, cohesive system.

The core problem is to design and build a robot that can autonomously execute a full “sense-plan-act” pipeline. The robot will explore an unknown arena, building a map, and performing a perception-driven pick-and-place task. More specifically, it will retrieve coloured blocks and place them in corresponding bins in an unseen arena with obstacles, thus solving a fundamental systems integration challenge.

1.3 Project Objectives

The primary objective of this project is to design and verify a mobile manipulator system—comprising a Leo Rover, manipulator, depth camera and LiDAR—capable of operating autonomously in an unknown environment. Specifically, the robot must:

- **Navigate and Map:** Scan the environment to build a representation for navigation while avoiding obstacles and remaining within mission boundaries.

- **Identify and Sort:** Use computer vision to locate coloured target blocks (e.g., red, blue, green) and distinguish them from obstacles.
- **Manipulate:** Plan paths to approach targets, precisely grasp blocks weighing up to 0.2kg, and transport them to the corresponding collection bins.
- **Operational Constraints:** Complete the entire sorting mission within 20 minutes using on-board power, subject to structural rigidity and safety compliance.

1.4 Responses to Feedback from Preliminary Design Review

Feedback of System:

- **Feedback 1:** *Diagram is good. It would benefit from more color coding and different box shapes.*
Addressed: We have updated the system block diagram (Fig. 1) with distinct color coding to clearly differentiate between the various sub-modules and electrical domains of the rover.
- **Feedback 2:** *Justifications would benefit from clearer links to requirements.*
Addressed: We have comprehensively updated the Component Selection section (Table 1). The justification for each hardware choice now explicitly maps to the specific Functional and Performance Requirements (e.g., Req. 1.1.1, Req. 1.5) detailed in Appendix A.3, ensuring strong traceability between system design and mission objectives.

Feedback of Power:

- **Feedback 1:** *The estimated battery power output seems very high. Are you sure the voltage is correct? It is 3s battery and each cell is roughly 4 volts, giving 12 volts. Thus it is not enough.*
Addressed: In PDR we explained that the 3S battery is not enough for the rover, thus we replaced it with a battery with larger capacity and a power distribution board. In FDR we update with the two Anker Prime Power Bank and the original 3S battery. Now the power is enough.

Feedback of CAD:

- **Feedback 1:** *General 3D renders shown but no specific dimensions.*
Addressed: Mechanical design section has been updated to include engineering drawings of both the assembly and custom mounting platform. This shows key assembly dimensions alongside the positioning of components on the raised level.

Feedback of Analysis:

- **Feedback 1:** *The language would benefit from being more specific and more details, including specific testing methods.*

Addressed: The Analysis section has been revised to use more specific and technical language. Verification Method statements now describe more clearly how each test is conducted and what is measured.

- **Feedback 2:** *The tests are achievable although they do not have specific timeframe, test after how many trials is considered completed?*

Addressed: The Verification Method for each requirement has been updated to include a clearer test scope, such as repeated trials or defined test duration, so that each test has a more explicit completion condition.

- **Feedback 3:** *Parts and software is generally not linked to requirements (with exception of manufacturing).*

Addressed: The Analysis section has been revised to improve requirement traceability. The relationship between system functions, software behaviour, and hardware implementation is now stated more clearly within the relevant analysis paragraphs.

- **Feedback 4:** *The success criteria could be more specific for most of designed tests.*

Addressed: A Success Criteria statement has now been added to each requirement in the Analysis section. These criteria define clearer and more measurable conditions for successful test completion.

For the remaining sections in the Preliminary Design Review, we did not receive further negative feedback or suggested changes.

1.5 Summary of Modifications Since Preliminary Design Review

Sustainability: Added new sustainability checklist in Section 2.

Cybersecurity: Added new section discussing cybersecurity considerations in Section 3.

Mechanical Design: Design updated to integrate new energy storage on new mounting brackets over the wheel axles. Drawings added for clarity of robot dimensions and specific positioning of assembled components.

Power: Updated to account for new power sources — two Anker Prime Power Banks.

Software: Added ROS2 RQT graph in Section 7.1 (see Fig. 9). *Note: since the current codebase is still undergoing integration, we present the RQT graph for the current simulation setup, which uses ‘stub’ nodes. These ‘stubs’ use the same ROS2 topic and action interfaces as the ‘real’ nodes, but directly modify the simulator state rather than performing true perception/interaction. As integration proceeds, we will replace stubs with their real counterparts.*

- **Computer Vision:** Added more details about how YOLOv5 runs and calculating positions of recognized objects in Section 7.1, with update on newly implemented smoothing and voting method to improve the stability and reliability of recognition results.
- **Navigation:** More detail on target localisation, transform caching and cost map configuration in Section 7.1.

Analysis: The Analysis section has been revised to improve clarity, specificity, and traceability to requirements while keeping the original technical content and structure.

- **Verification Method:** Each requirement now includes a more specific Verification Method. Repeated trials, representative test runs, or defined observation periods have been added where appropriate.
- **Success Criteria:** A new Success Criteria subsection has been added for each requirement. This provides a clearer definition of successful performance, including measurable limits such as trial success rate, timing, clearance, and allowable error where relevant.

Project Plan: Updated the timeline and clarified the division of responsibilities.

2 Sustainability Checklist

Materials As discussed in 5.3.1, additional components have been manufactured via FDM 3D printing with PLA and laser-cut acrylic panels. As a compostable bioplastic, PLA is a more sustainable choice than other plastics like ABS. Acrylic is not biodegradable, but can be commercially recycled. Where metal components are necessary, mass produced, off-the-shelf items are used, minimising the need for energy intensive machining processes such as CNC milling or turning. While the mechanical design presented is considered in the context of this prototype, custom parts have been designed with mass manufacturing methods in mind using similar or identical materials. Injection moulding would provide a suitable alternative to 3D printing. For uniplanar parts, stamped sheet metal would be a scalable, robust and cost-effective manufacturing method. Designs have also been thoughtfully considered to avoid the need for superfluous iterative prototyping which would waste material and energy.

Software The largest source of power draw in the software architecture is due to the use of YOLOv5, a deep neural network for multi-object classification. To reduce software overhead, we use smoothing and filtering to enable message publishing at a lower framerate, reducing unnecessary downstream computation and communication. Additionally, we reduce the training cost of our vision solution by fine-tuning a pretrained YOLO model rather than training from scratch. Other design decisions, such as the use of LiDAR over visual SLAM and disabling the redundant Leo Rover camera further reduce unnecessary computational load.

Energy The primary factors affecting energy efficiency, such as selection of motor components and weight minimisation, are largely constrained by the given components used. However, we manufacture a lightweight payload sled to avoid increasing the weight of the robot (and hence power spent on locomotion). The energy sources are the Leo Rover battery and two Anker Prime power banks. These energy sources are rechargeable, and can be reused in future. As noted in the Emissions section, batteries are charged using the University of Manchester power network, which is renewably sourced.

Waste At the end of the robot's use for our project, the main components and rover will be reusable for future projects. However, the plastics used in the payload sled will be discarded, constituting the primary source of waste. As mentioned in the section on Materials, we use recyclable materials such as PLA and Acrylic. To minimise plastic waste, we prioritised material efficiency in the payload sled design, and all plastic materials will be recycled after disassembly.

Emissions Before operation, the primary source of emissions is manufacturing of 3rd party components such as the Leo Rover, MyCobot arm and RealSense depth camera. Few manufacturers publish granular emissions data, so it is challenging to identify opportunities for emissions reduction by purchasing different components. We seek to amortise the cost of manufacturing via modular design; each component can be reused and repurposed for future projects, ‘spreading’ the cost of manufacturing over many years of use. In operation, most emissions are due to electricity drawn from the University of Manchester. As the University uses a mixture of direct solar power¹ and renewable energy purchases backed by REGO Certifications,² this is a low-emission electricity source.

Communications Communications are minimal, except for minor text logging over SSH for remote monitoring of the robot state. We avoid large data transfers such as images or other sensor data. Additionally, the computationally-heavy vision and SLAM workloads are processed locally on the NUC, rather than calling out to off-board servers.

Modularity The design is highly modular. Sensors and effectors are mounted to the rover chassis in a non-destructive fashion, so they can be swapped in case of failure. All components except the payload sled are off-the-shelf 3rd party components, so they can be replaced or repurposed for other projects as needed.

Location/Placement The robot is deployed within the same building it is stored and developed, so sustainability concerns around deployment are minimal. Transportation only involves carrying the robot between its storage location and the laboratory, so neither location nor transport are significant sustainability concerns for our use case.

Maintenance The mission duration for the robot is short (20 minutes), so mid-mission maintenance or live diagnostics are unnecessary to achieve the stated design requirements.

Repurposing All additions made in the design are non-destructive, so the robot and components can be easily reused, continuing its educational use case. Additionally, individual components such as the arm or LiDAR can be easily removed from the robot and reused for other projects.

¹<https://renewableenergyinstaller.co.uk/2025/09/flagship-71mw-enviromena-solar-farm-goes-live-in-esssex-backed-by-uks-largest-educational-cppa>

²<https://www.mub.eps.manchester.ac.uk/mace/2021/01/25/university-commits-to-100-renewable-energy>

3 Cyber Security Considerations

The system has been designed to run on a local wired network, interfacing all devices over Ethernet. Remote access is accommodated for mission start/stop and for limited diagnostic readout. This remote access is facilitated over a local Wi-Fi network hosted by the LeoRover base. Control is then achieved through Secure Shell (SSH) commands to run ROS2 launch files.

Access to the local Wi-Fi network and to the SSH terminal are both password protected. This permits only authenticated users to access and send control commands to the robot. The local network access method avoids exposure to the public internet, requiring physical proximity to even attempt accessing the robot remotely. Additionally, the team codebase is stored in Git repository which requires authentication modify, permitting only authorised users to view or edit onboard software. Ideally this repo would be private, but it is public for marking purposes.

4 System

This section outlines the high-level system architecture and key components required to achieve the objectives.

4.1 System Architecture

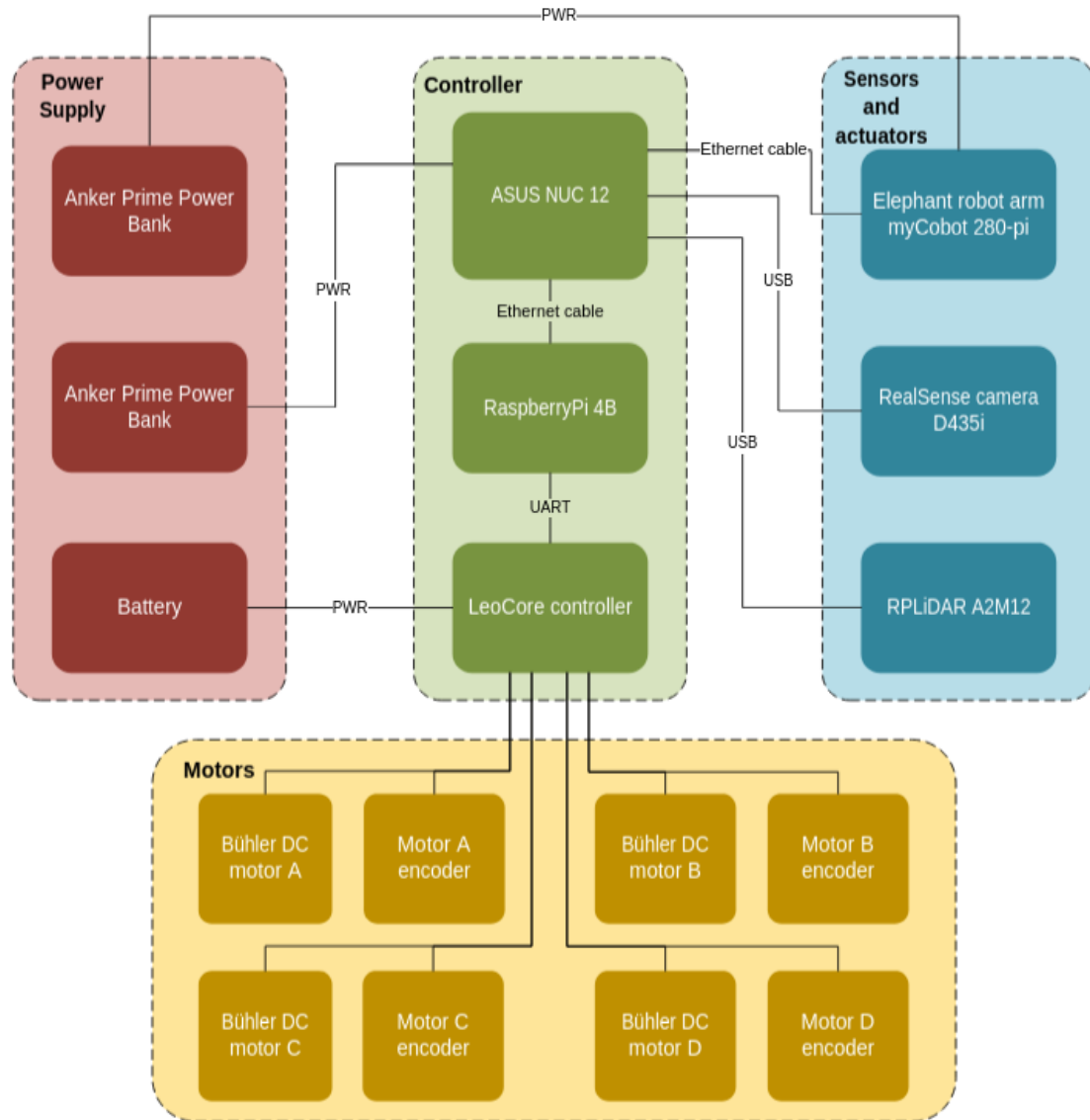


Fig. 1. System block-diagram of all the hardware components on the robot. Components that are provided but not used so far like IMU, onboard RGB camera and Antenna are not included into this diagram.

The system architecture adopts a hierarchical control strategy. The ASUS NUC 12 Pro serves as the high-level computing unit for SLAM and computer vision, processing inputs from the LiDAR and Depth Camera. It communicates via Ethernet with the Raspberry Pi 4B, which acts as a bridge to the low-level LeoCore controller for real-time motor actuation.

4.2 Component Selection

Table 1. System Component Description and Justification

Component	Description	Justification
Mobile Platform (Leo Rover)	A 4-wheel drive, rugged mobile chassis powered by 4x Buehler DC motors (1.61.077.414).	Provides the necessary mobility to traverse the arena at required speeds up to 250mm/s (Req. 1.1.1). The planetary gear-boxes provide high torque for the payload, while its base weight ensures the integrated system remains under the 20kg safety limit (Req. 3.1.3).
Manipulator (myCobot 280-pi)	A 6-DOF collaborative robot arm from Elephant Robotics, mounted on the front chassis.	Directly fulfills the manipulation requirements (Req. 1.5). Chosen specifically for its capacity to grasp blocks up to 0.2kg (Req. 1.5.1) and its kinematic reach to cover the required 250x250mm operational plane at the specified height (Req. 1.5.2).

Component	Description	Justification
ASUS NUC 12 Pro	The central processing unit (Model NUC12WSK) hosting the ROS 2 master, SLAM, and vision algorithms.	The core computing unit required to achieve full autonomy (Req. 2.3). Its high processing power is strictly required to simultaneously handle real-time LiDAR SLAM for collision avoidance (Req. 1.2) and deep learning vision models for target identification (Req. 1.3, 1.4).
Raspberry Pi	The bridge computer connecting the NUC and the LeoCore via serial communication.	Serves as the crucial communication bridge. Necessary to manage hardware interfaces and ensure real-time command transmission to the wheel motors for accurate autonomous movement (Req. 1.1).
LeoCore Control Board	The dedicated STM32F4 microcontroller board for the Leo Rover chassis.	Acts as the hardware driver ensuring real-time PID control of wheel motors to meet movement specifications (Req. 1.1) and executes the emergency stop halt commands to fulfill safety compliance (Req. 3.1.2).

Component	Description	Justification
LiDAR Sensor (RPLiDAR A2M12)	A 360-degree 2D laser scanner with 12m range.	Essential for robust obstacle avoidance (Req. 1.2). Specifically selected for its range to guarantee the detection of walls and obstacles within the required 1000mm radius (Req. 1.2.1).
Depth Camera (Intel RealSense D435i)	A depth camera capable of capturing both RGB images and depth point clouds.	The primary sensor for perceiving the environment. It is required to identify the color and pose of target blocks (Req. 1.3) and destination bins (Req. 1.4), providing the essential depth coordinates (x, y, z) needed for successful grasping (Req. 1.5).
Anker Prime Power Banks (x2)	Two portable high-capacity power banks, each capable of providing up to 140W output via a single USB-C port.	Solves the power deficit of the standard battery. These provide dedicated, sufficient power to the high-load components (ASUS NUC and myCobot manipulator), allowing the robot to complete the mission autonomously without external tethering or recharging (Req. 2.1).

Component	Description	Justification
Leo Rover Battery	The standard 11.1V 3S battery included with the mobile platform, capable of an 88.8W max output.	Provides sufficient power to drive the Leo Rover chassis, wheel motors, encoders, and the LeoCore controller for standard mobility and low-level communication.
<i>RGB Camera (Unused)</i>	The standard 5MPx fish-eye camera embedded in the Leo Rover front plate.	Not Utilized. Lacks depth sensing capabilities required for 3D manipulation. It is functionally replaced by the RealSense D435i.
<i>External Antenna (Unused)</i>	The standard 2.4GHz external WiFi antenna mounted on the rover.	Not Utilized. Redundant. High-bandwidth communication is handled directly by the NUC's internal WiFi 6E module.
<i>Internal IMU (Unused)</i>	Grove 9DOF IMU v2.0 integrated with the LeoCore.	Currently Not Utilized. Primary navigation relies on LiDAR-based SLAM and wheel odometry. IMU fusion is reserved as a backup strategy.

5 Mechanical Design

This section discusses the range of considerations made when designing the assembly of additional components, modifying the LeoRover base platform. Renders of the 3D CAD model are included alongside discussion of the placement of each component chosen in Table 1. The key areas where structural integrity must be considered are then reviewed, followed by verification of the robot's stability.

5.1 3D SolidWorks Model

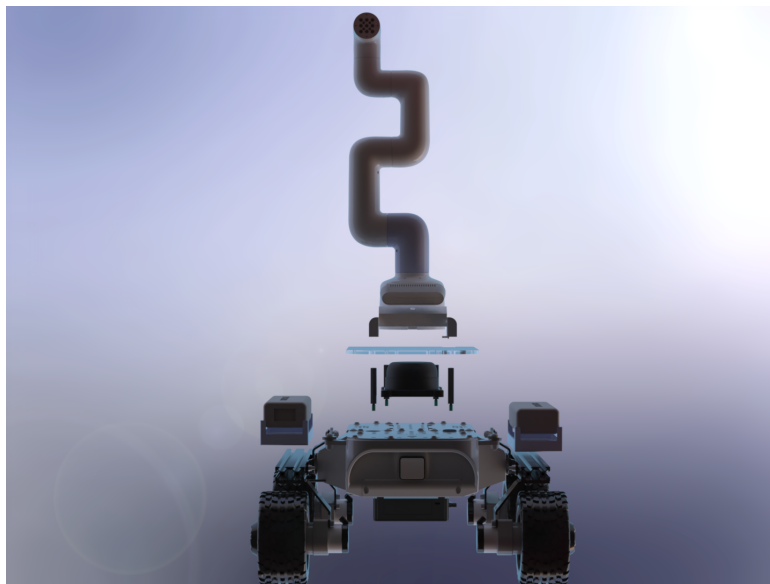


Fig. 2. Front Exploded View. End-effector model not included.

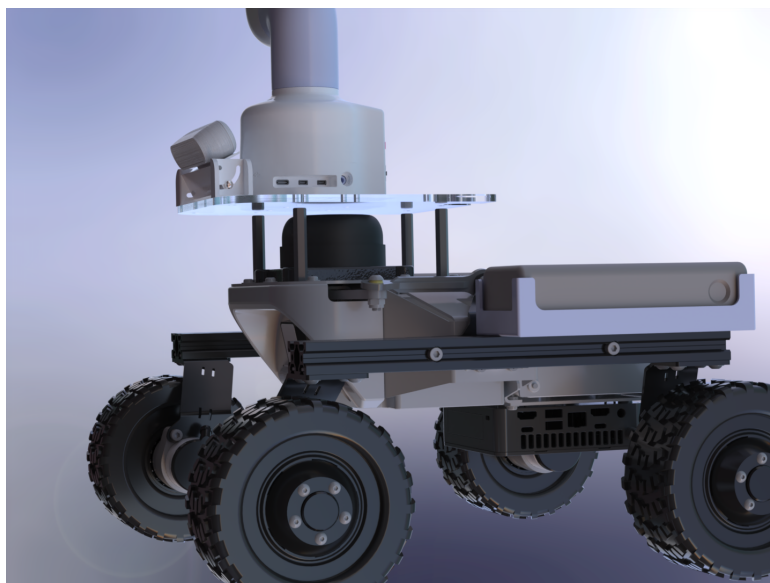


Fig. 3. Payload Sled. Intel NUC located on robot underside.

D

D

C

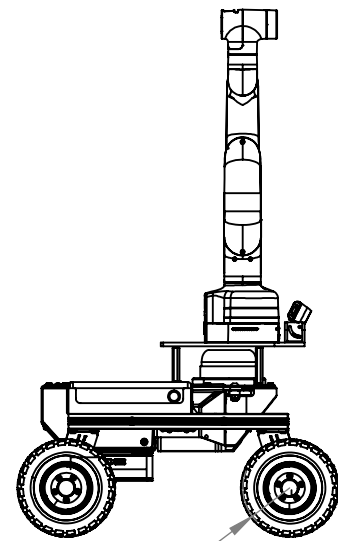
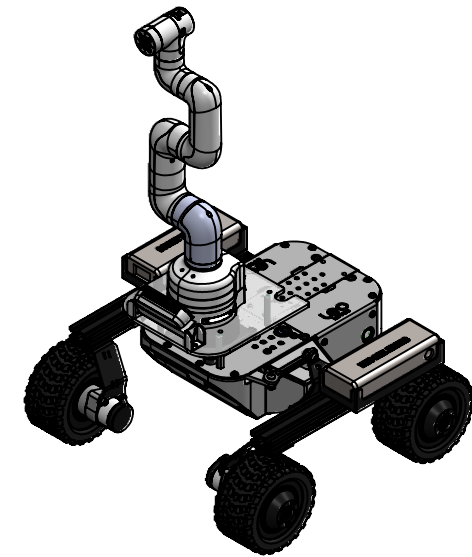
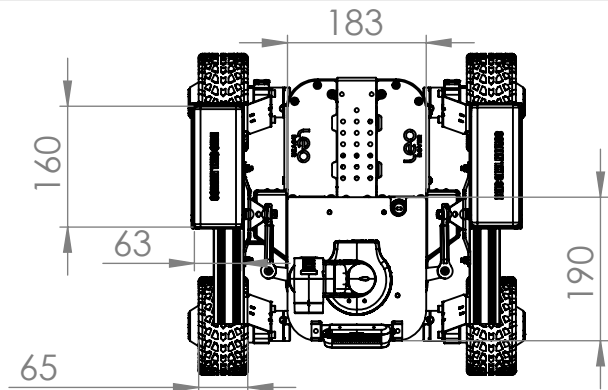
C

B

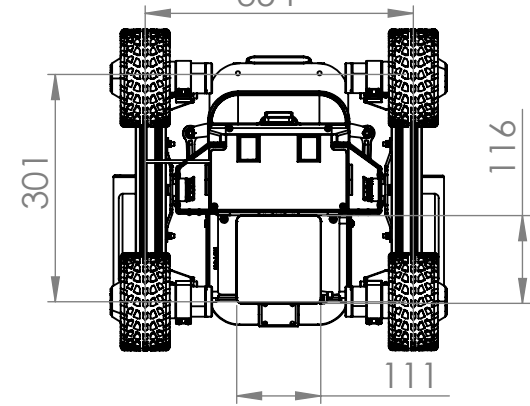
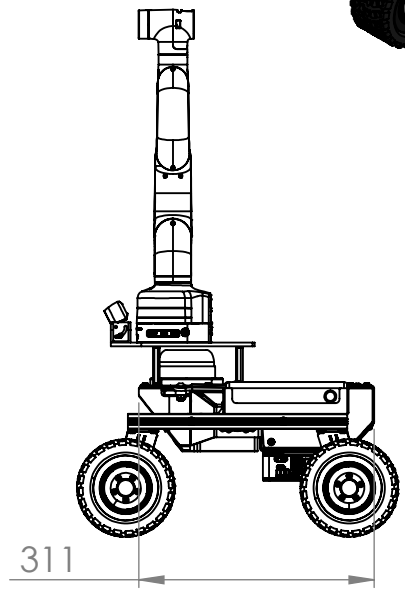
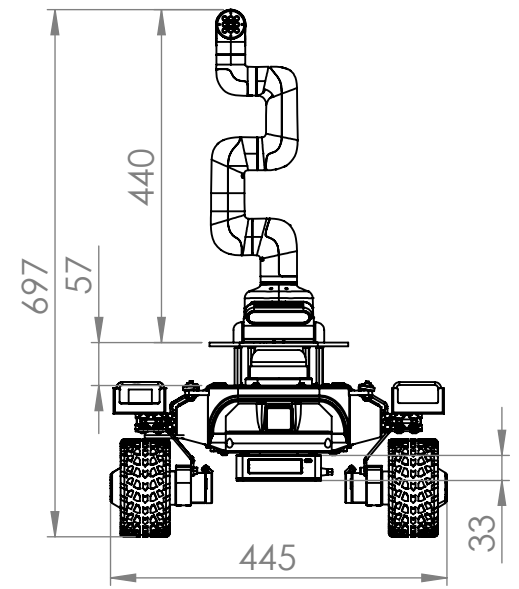
B

A

A



R62



DO NOT SCALE DRAWING		REVISION
Drawn by: Mike Fahey		
TITLE: Team 10 Robot Assembly Drawing		
DWG NO. T10-ASM-001 (Robot Assembly)		A4
SCALE:1:10		SHEET 1 OF 1

D

C

B

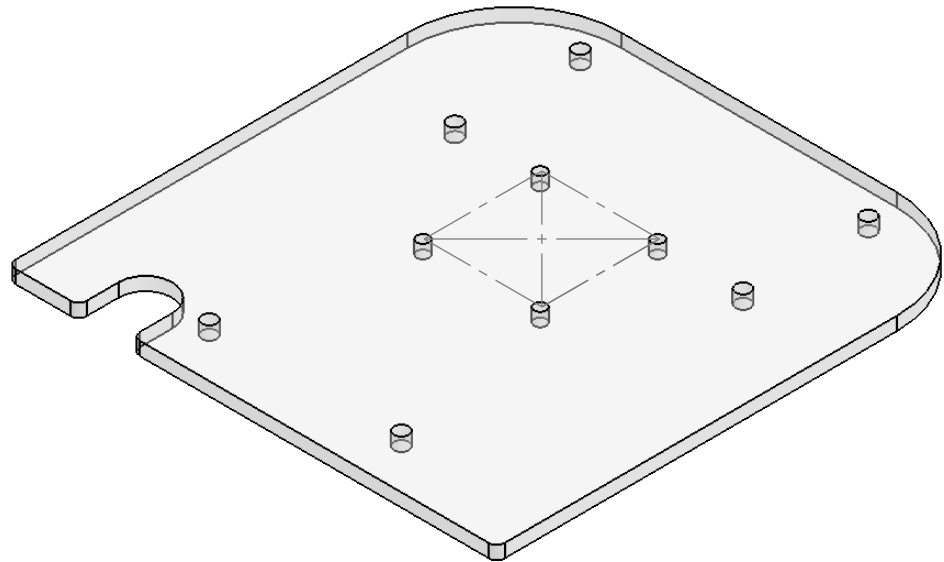
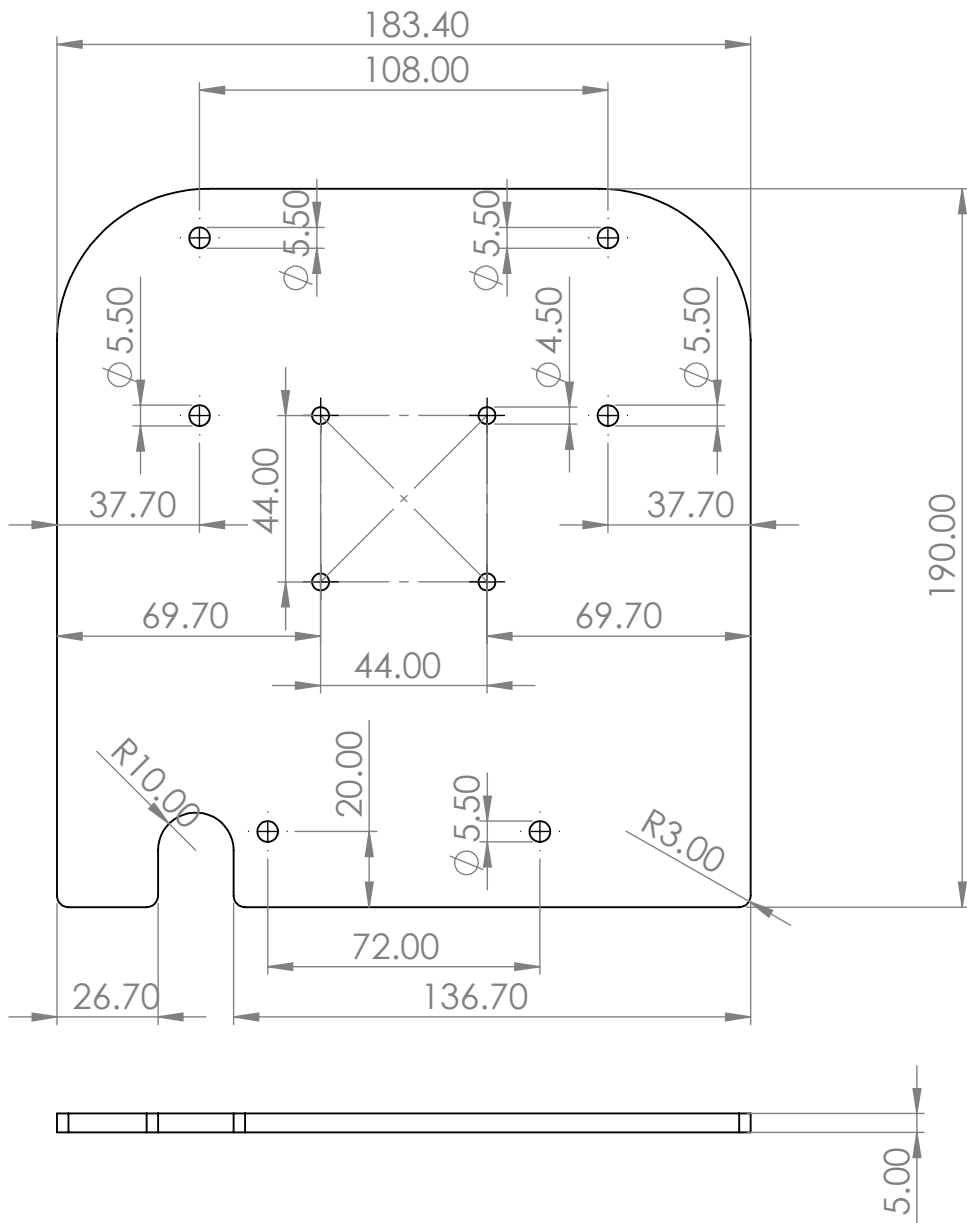
A

D

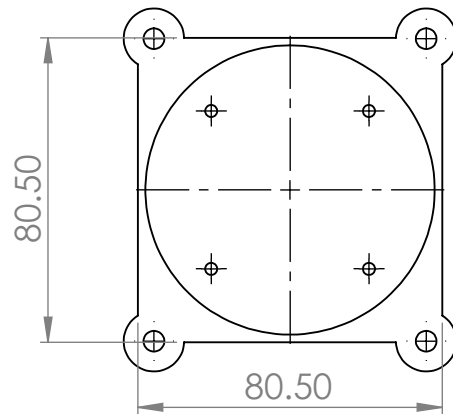
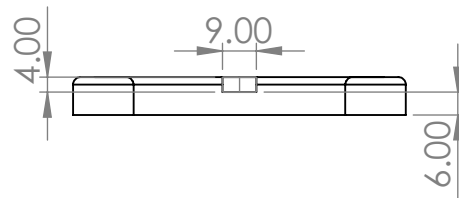
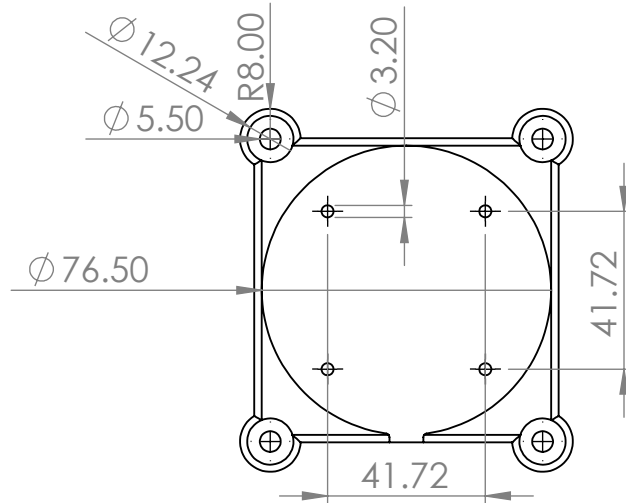
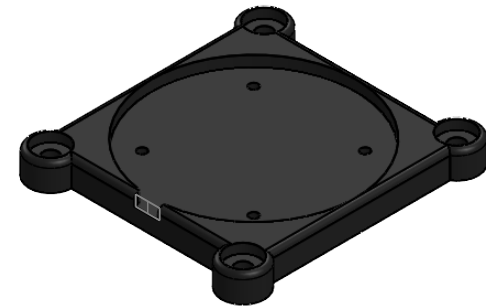
C

B

A



DO NOT SCALE DRAWING		REVISION	
Drawn by: Mike Fahey			
TITLE: Team 10 Manipulator Platform Drawing			
DWG NO. T10-PRT-002 (Arm Platform)			A4
SCALE:1:2		SHEET 1 OF 1	



DO NOT SCALE DRAWING		REVISION
Drawn by: Mike Fahey		
TITLE:		Team 10 Lidar Mount Drawing
DWG NO.	LidarBaseMount	A4
SCALE:1:2		SHEET 1 OF 1

Table 2. Payload Sled Subsystem Requirements

#	Requirement	Justification
1	Component placement and mounting shall avoid obstructing the LiDAR plane.	The LiDAR available has a 2D planar scan area which the robot will be dependent on for navigation and mapping. Occlusions reduce the robot's ability to fully map its surroundings and so should be minimised.
2	Component placement and mounting shall avoid obstructing the depth sense camera field of view.	While block and bin localisation through computer vision is more robust to occlusion than LiDAR mapping, a clear field of view is still essential to permit efficient localisation of targets.
3	Component placement and mounting shall avoid obstructing the reach of the manipulator arm.	The available manipulator has 6 degrees of freedom which enables reach of all x, y, z coordinates within a radius. Objects in this radius require constraints to be put in place to prevent the manipulator from colliding with any part of the robot. These constraints limit the manoeuvrability of the manipulator. These limitations should be minimised to avoid inaccessible block poses.
4	Component placement and mounting shall result in the robot's centre of mass falling inside the trackwidth-wheelbase bounding box in all modes of operation.	To guarantee stability and avoid the possibility of the robot tipping, the robot's centre of mass needs to remain within the footprint of its wheels.
5	Camera placement shall permit a view of the top of target bins and block platforms.	Manipulation is guided by poses recovered from depth camera imagery. For accurate pose recovery, a view from above the $x - y$ plane which these targets occupy is required.
6	Manipulator placement shall permit gripper reach of all possible block poses upon their platforms.	The exact location of a block upon its platform is undetermined. To collect all blocks, the manipulator must be able to reach the extremities of possible locations when the robot is within the closest feasible range of each platform.

5.2 Component Placement Challenges

The LeoRover Platform offers a number of mounting points on its top surface. This allows excellent flexibility in the custom payload sled design, with respect to its secure attachment to the base mobile platform. The design of the payload sled, and more specifically the organisation of components, is therefore guided primarily by the subsystem requirements in Table 2, derived from the following key goals:Repurposing

- Minimise occlusion of robot sensors.

- Maximise manipulator reach and range of motion.
- Maximise platform stability.

The myCobot280 manipulator has a maximum reach of 280mm from its centre point, shown in Fig. 4. Even with the additional reach provided by the gripper, it is very possible to violate requirement 6. It is known that block platforms are 210mm tall which can be approximated to being the same height as the top surface of the LeoRover platform. This gives two guides to constrain placement of the arm: that it should be mounted as far forward as possible, while maintaining stability; and that it should be as low as possible. Mounting components above the arm's reach radius is not feasible, so adequate space must be left below the arm for other components.

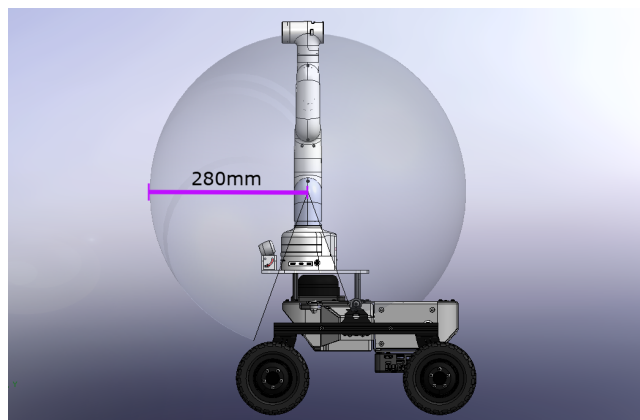


Fig. 4. Reach radius of mounted manipulator

Requirement 1 prevents placement of components in the LiDAR plane. Two options were considered for mounting the LiDAR; initially, placement on the underside of the robot was preferred in order to free up space on the platform's top surface for the manipulator. However, the occlusion of the LiDAR plane by the robot wheels mean that this approach would seriously reduce mapping performance. Instead, the current solution of mounting the manipulator base on an acrylic platform above the LiDAR was chosen. Minimising the height of this platform has been a key design priority, resulting in an extremely low-profile LiDAR base mount. Thin sheet metal platform scaffolds (to minimise LiDAR occlusion) were considered, but eventually abandoned to reduce manufacturing complexity. The team is opting for off-the-shelf hexagonal stand-offs to support the platform instead.

Due to the flat environment in which the robot will operate, we have decided to mount the Intel NUC on the underside of the robot, towards the rear. Cabling can be routed through gaps in the LeoRover chassis, so this does not limit inter-component connections. In addition, rearwards placement of the NUC moves the centre of mass of the system to counterbalance the manipulator when it reaches forward.

The space remaining on the top surface of the robot provides ample space for integration of addi-

tional components in future such as a resting tray to secure a collected block for safer transit.

The additional energy storage used, as required by 6.2, are Anker Prime Power Banks and have been integrated into the mechanical design of the platform by mounting to the aluminium extrude axles. A PLA additively manufactured bracket is used to retain a battery over both axles. These are designed to be friction fit with the batteries so that they can be easily removed for charging and safe storage.

5.3 Stability & Manufacturability

5.3.1 General Considerations

Due to the nature of the mission, dynamic loading cases are not significantly greater than when the robot is static. The robot is constrained to a steady $250mm/s$ navigation speed, and acceleration forces are not expected to exceed $0.5m/s^2$. Because of these small dynamic forces, if mountings are strengthened to support the weight of each component in static load (with a minimum safety factor of 1.5), they will be adequate to support those components whilst the robot is moving.

These minimal loading forces reduce the requirements for material choices; therefore the use of plastics for self-designed components is sufficient. For multiplanar parts, FDM 3D printing is the most suitable method of manufacture, allowing excellent flexibility in geometry. The mission will occur at room temperature, so polylactic acid (PLA) is a suitable filament choice. For uniplanar parts, 2D CNC methods are available and so laser-cut acrylic is chosen.

5.3.2 Manipulator Platform Structural Analysis

The platform which the manipulator sits on is subject to the greatest. Finite Element Analysis is employed to verify that the designed part is strong enough to support the manipulator. The simulation was setup loading the mass of the manipulator across the area enclosed by its 4 mounting holes. 5mm Acrylic sheet has been chosen for this part and the simulation in Figure 5 shows minimal stresses. The part should therefore be capable of withstanding all loading cases imposed by the motion of the arm.

5.3.3 Robot Stability and Centre of Mass Analysis

Assumptions:

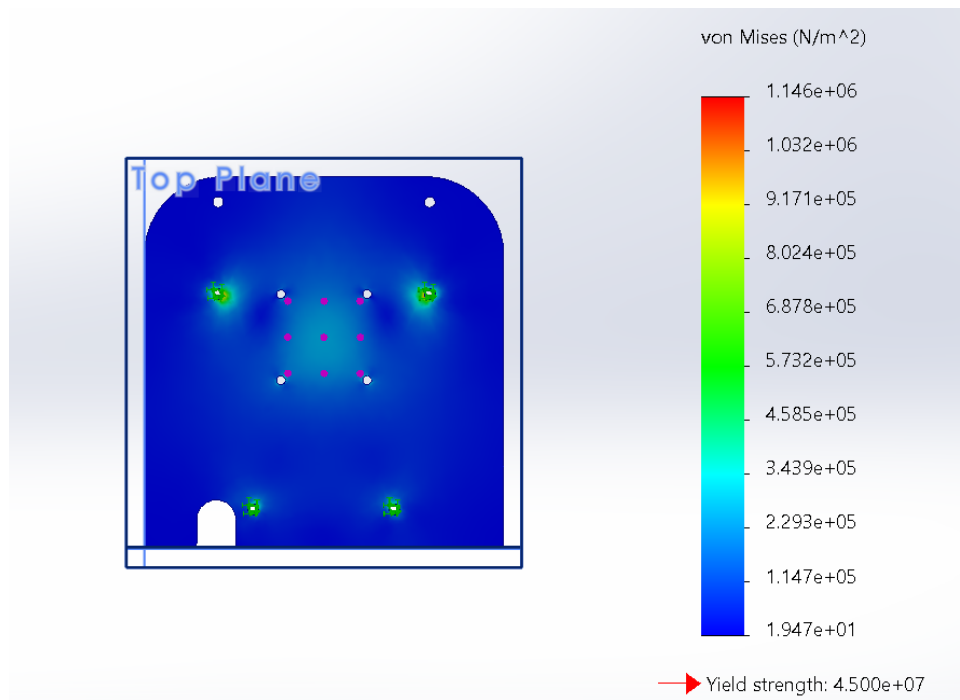


Fig. 5. Finite Element Analysis of Acrylic Manipulator Platform showing minimal stresses when loaded with weight of the manipulator over its mounting holes.

1. **Manipulator is rigid and uniform.** Whilst this isn't strictly true (it's base heavy), assuming uniformity allows calculation of a worst case for where the manipulator's centre of mass may fall.
2. **Manipulator is fully outstretched.** Also a worst case, finding the furthest forward the centre of mass may fall.
3. **LeoRover, NUC and LiDAR are all uniform and co-planar.** Their individual centre of masses are found at their object centres and all of these are on the plane which cuts the robot down its centreline.
4. **Depth Sense Camera, block payload and all plastic mounts have negligible mass.**

To verify that the centre of mass of the system will never fall outside the robot footprint, the above assumptions are made to calculate the worst-case location. An approximate treatment shown in Fig. 6 shows that the centre of mass moves very little from the centre of the LeoRover platform. This is not surprising; most of the system's mass is in the chassis of the mobile platform. The rover's long wheelbase and wide track width means that a significant moment would need to be applied to tip the robot. It is reasonable to assume that no such moment will be experienced in the operation of the robot, giving confidence in its stability.

The calculation for this worst case position is as follows:

$$8.2g \begin{pmatrix} \bar{x} \\ \bar{z} \end{pmatrix} = 0.5g \begin{pmatrix} 105 \\ 90 \end{pmatrix} + 6.6g \begin{pmatrix} 210 \\ 100 \end{pmatrix} + 0.2g \begin{pmatrix} 350 \\ 220 \end{pmatrix} + 0.9g \begin{pmatrix} 490 \\ 260 \end{pmatrix} \quad (1)$$

$$\begin{pmatrix} \bar{x} \\ \bar{z} \end{pmatrix} = \frac{1}{8.2} \left[\begin{pmatrix} 52.5 \\ 49 \end{pmatrix} + \begin{pmatrix} 1386 \\ 660 \end{pmatrix} + \begin{pmatrix} 70 \\ 44 \end{pmatrix} + \begin{pmatrix} 441 \\ 234 \end{pmatrix} \right] \quad (2)$$

$$\begin{pmatrix} \bar{x} \\ \bar{z} \end{pmatrix} = \begin{pmatrix} 238 \\ 120 \end{pmatrix} \quad (3)$$

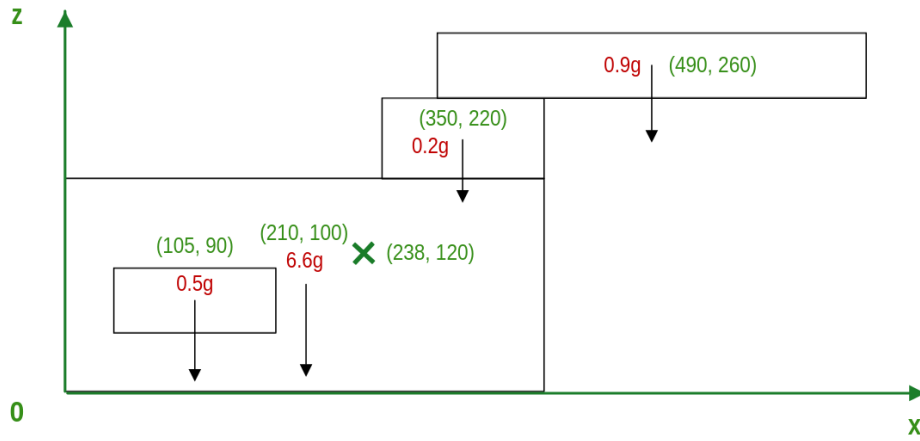


Fig. 6. Centre of Mass Approximation

6 Electrical Design

In this section we discuss the robot electrical design. The power connection diagram for all the peripherals on the robot is illustrated in the Section 6.1, and the power budget of the robot is calculated with the explanation of our choice of battery in the Section 6.2.

6.1 Power Connection

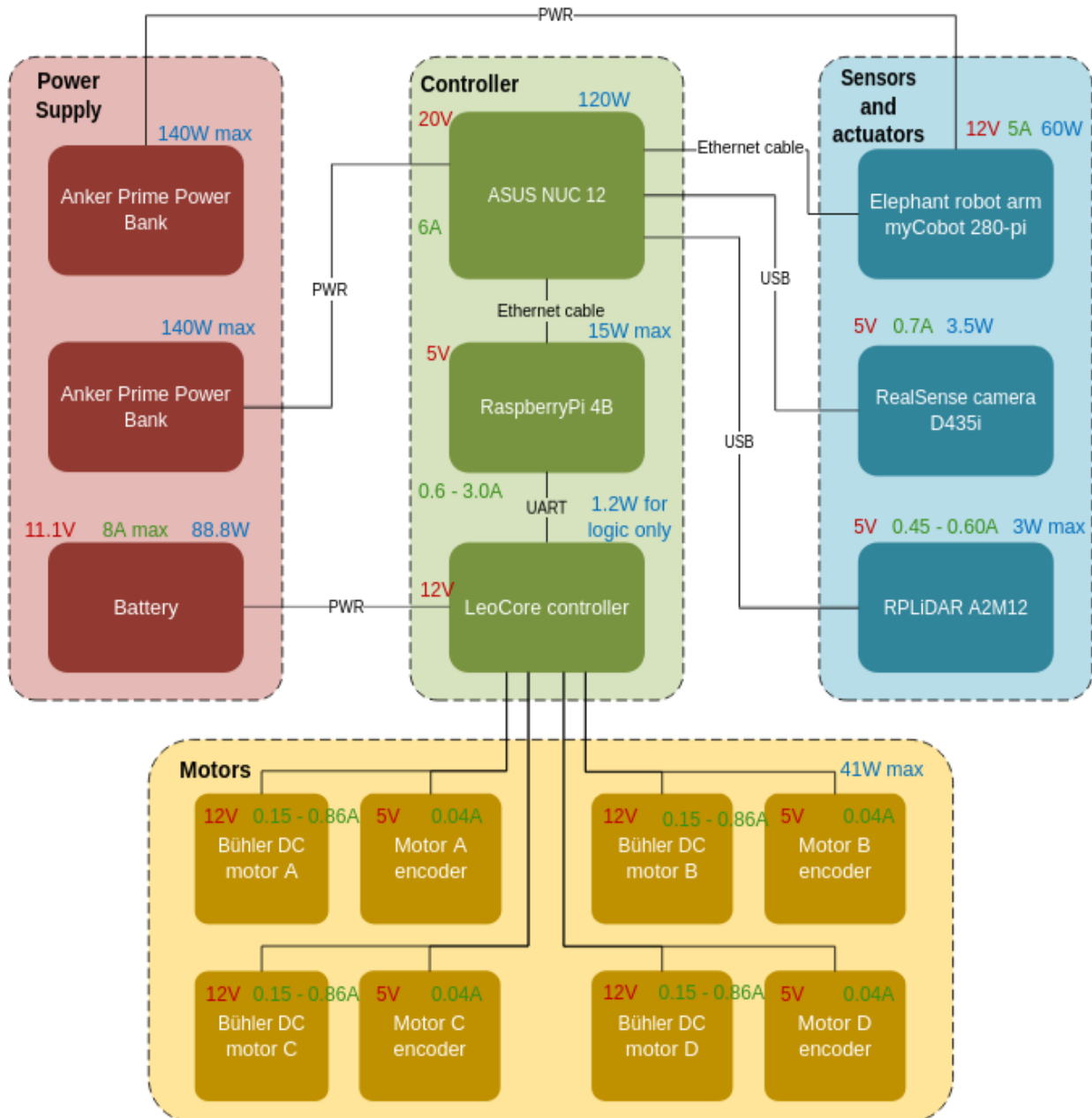


Fig. 7. Power connection diagram for the peripherals on the robot. For each component module, the voltage, current and power are marked in red, green and blue respectively. Components that are provided but not used so far like IMU, onboard RGB camera and Antenna are not included into this diagram.

The power connection diagram of the robot is illustrated in Fig. 7, where the estimated voltage, cur-

rent and power consumption for each peripheral or module is marked. As discussed in Section 4, the robot peripherals we use are power supply module, motors and encoders for wheels, NUC, Raspberry Pi, LeoCore controller, Elephant robot arm, RealSense camera and RPLiDAR, with the provided onboard RGB camera and antenna not in use. Thus, although the RGB camera and antenna are included in the hardware design of the LeoRover, they are not included in the power connection diagram and the power budget calculation.

6.2 Power Budget

Components	Voltage(V)	Current(A)	Power(W)
Battery	11.1	8 max	88.8 max
2 Anker Prime Power Bank	/	/	300, 140 max single port
LeoCore Controller Logic only	12	0.1 approx	1.2 approx
RaspberryPi 4B	5	0.6 - 3.0	3 - 15
4 Wheels Buehler Motors 1.61.077.414	12	0.60 - 3.44	7.2 - 41.28
4 Wheels Pololu Romi 12 CPR magnetic encoders	5	0.04 approx	0.2 approx
ASUS NUC PRO-NUC12WSK	20	6	120
RPLiDAR A2M12	5	0.45 - 0.60	2.25 - 3.00
Intel RealSense D435i	5	0.7	3.5
Elephant Robotics arm myCobot 280-pi	12	5	60

Table 3. Power budget of each components based on datasheets, test results online and approximation. Many current consumptions are given as ranges in this table. For example, for RaspberryPi, 0.6A is the typical bare-board active current consumption and 3.0A is the recommended PSU current capacity.

The power budget for the peripherals on the robot is shown in Table 3. The maximum power consumption of all the peripherals is

$$1.2 + 15 + 41.28 + 0.2 + 120 + 3 + 3.5 + 60 = 244.18(W),$$

and since the RPLiDAR and RealSense camera are powered by the NUC, their power consumption is included its 120W max power consumption. Thus the maximum total power consumption of the robot is

$$1.2 + 15 + 41.28 + 0.2 + 120 + 60 = 237.68(W)$$

The power source we use are the battery of LeoRover and two Anker Prime Power Bank. The max power output of the battery is 88.8W max, and the power bank has 140W max output for single USB-C port, 300W in total. The overall power of power source is

$$88.8 + 2 \times 300 = 688.8(W)$$

Since the two power bank each supplies power to NUC and myCobot robot arm, and the rest of the

rover is supplied by the battery, we have the safety margin for NUC, myCobot and rest of the rover for (calculated using 140W max output for single USB-C port of the power banks):

NUC:

$$(140 - 120) \div 140 = 14.3\%$$

myCobot:

$$(140 - 60) \div 140 = 57.1\%$$

Rest of rover:

$$(88.8 - 57.68) \div 88.8 = 35.0\%$$

With such power safety margin, we have enough power to drive the rover and finishing tasks using NUC and myCobot robot arm. There are even surplus power of the two power bank as they can power three devices at once.

7 Software Design

The software component of the rover is founded on ROS2 for inter-component communication and coordination. A schematic of the software architecture is shown in Fig. 8. In order to improve software maintainability and debuggability, the architecture is modular; each ROS2 node has clearly-defined interfaces and messages which require no knowledge about the working models of other nodes. To ensure a consistent contract between nodes, ROS2 interfaces are pre-defined to minimise integration friction, described in Section 7.3. The GitHub repository can be found at <https://github.com/MJF2003/RSDPTeam10>. All custom software displayed in Fig. 8 will be run on the Intel NUC. The only software run on the Raspberry Pi will be the Leo Rover nodes which offer an interface with the wheel controls.

7.1 Software Architecture

Controller The architecture centres on a controller node which drives the system's behaviour. This node ingests and smooths block and bin observations from the computer vision node to generate a persistent list and track what has been collected. These observations are used to inform the control flow shown in Fig. 10 and define targets for navigation. The controller executes behaviour via ROS2 actions exposed by the manipulation and navigation nodes.

Computer Vision The computer vision node runs a fine-tuned YOLOv5 model to process depth camera inputs, classifying types of objects, recognizing attributes such as colours and shapes of them and calculating position coordinates based on the crops of recognized objects. It outputs the positions, colour and shape of blocks observed. It also outputs position and colour information for bins, and positions of the platforms on which blocks are placed. To guarantee that the outputs are stable and reliable for the other nodes to use, smoothing method and voting are implemented to smooth the positions and confidence, outputting results only with high confidence and enough votes on the attributes. The bin and block positions are consumed by the controller node and passed to the manipulator and navigation nodes for block retrieval and deposition. The platform poses are used by the navigation node: the LiDAR is positioned on top of the robot, above the level of the platforms, it does not perceive them—instead the platform positions are directly added to the costmap so they can be avoided during navigation.

SLAM The system uses Cartographer to perform SLAM and compute the map used in navigation. It subscribes to the LiDAR `/scan` topic and publishes the map frame to the `tf` tree.

Navigation The navigation node provides the controller node with a semantic, practical API via ROS2 actions. It serves as the interface for navigation planning and execution with Nav2, configuring key Nav2 parameters such as planning algorithms and cost map definitions. It also integrates platform information from the vision processing node, dynamically modifying the cost map to enable Nav2 to avoid areas of the platform that the LiDAR cannot detect. When there are no known obstacles to detect, this node also executes exploration logic to guide the robot's autonomous movement within the arena.

To ensure the continuity and reliability of transformation data, the system extends the TF buffer retention time to 10 seconds and introduces a timeout retry mechanism to address unstable publication frequencies, thereby striking a balance between response speed and data reliability. The system fuses real-time SLAM dynamic maps with high-frequency sensor data and employs a two-layer cost map strategy to balance global path optimality with local obstacle avoidance sensitivity: the global cost map is based on a static map and updated at a low frequency to reduce computational overhead, while the local cost map is updated at a high frequency using a rolling window to respond in real time to dynamic obstacles, ensuring navigation safety and flexibility. Upon receiving target coordinates, the system uses a TF2 transformation tree to automatically perform pose alignment between coordinate systems and drives the underlying path planning algorithm to find the optimal path within the real-time generated raster map; the entire navigation process is strictly controlled by behaviour tree logic, enabling the system to autonomously perform map cleaning, rotation detection, and obstacle avoidance manoeuvres when the path is blocked.

Manipulation The manipulation node is responsible for managing the state of the robotic arm and executing precise block grasping and homing sequences. Moving away from high-level wrappers, it directly utilizes the native ROS 2 Action Client API (`rclpy.action`) to interface with MoveIt 2 for advanced motion planning and execution. To ensure reliable and collision-free grasping, the node enforces strict spatial and orientation constraints on the end-effector during trajectory generation.

Furthermore, the system architecture completely decouples perception from manipulation. Rather than processing raw depth data internally, the manipulation node acts as a direct subscriber to the computer vision pipeline. It receives pre-calculated 3D target poses and confidence scores (`BlockPoseObservation`), dynamically filtering incoming data based on detection reliability and the arm's internal readiness state. This streamlined data flow allows the node to focus exclusively on robust trajectory execution and state management without the computational overhead of point cloud processing.

SOFTWARE BLOCK DIAGRAM

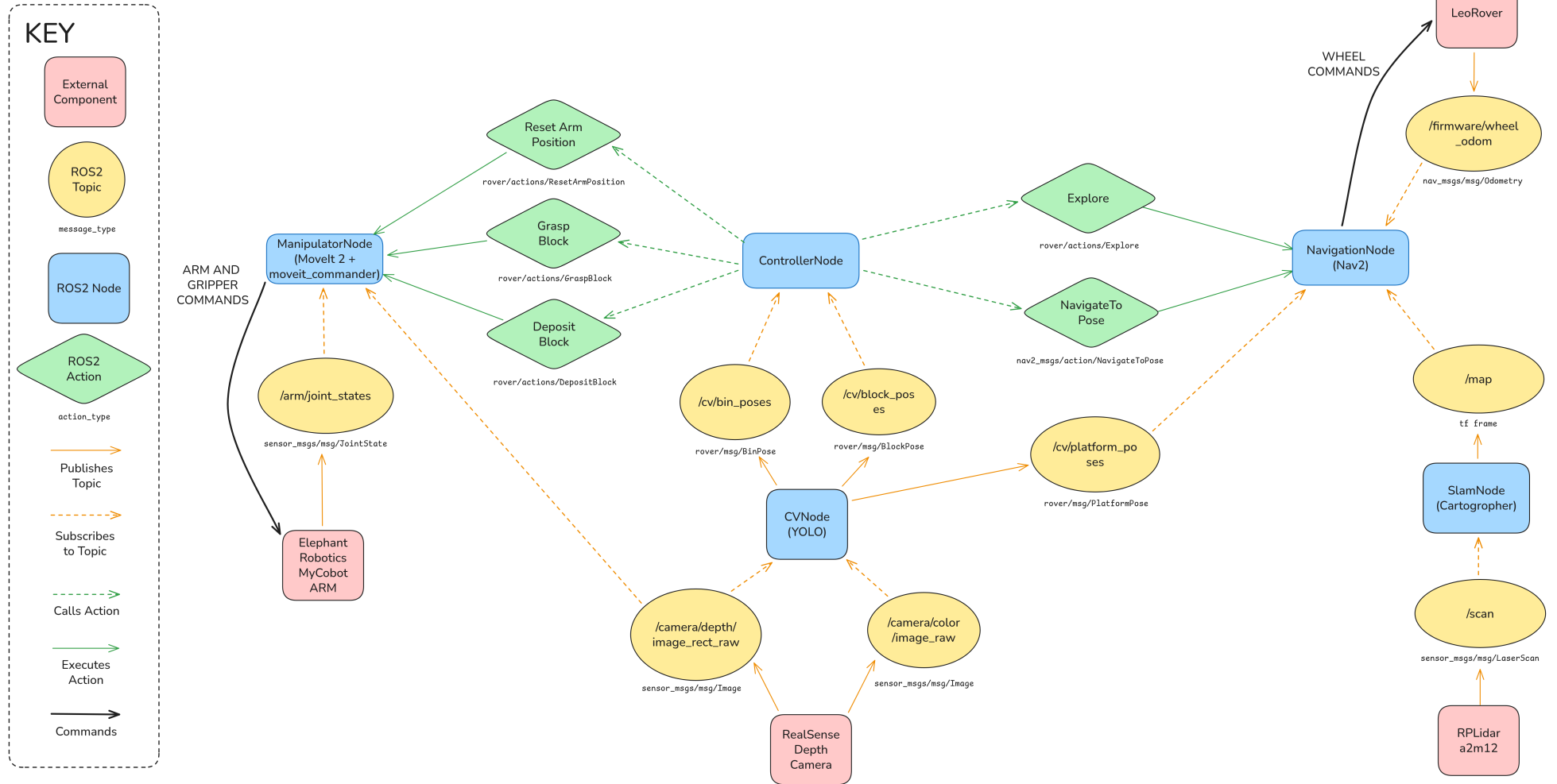


Fig. 8. Block diagram indicating the ROS2 architecture of the rover. To minimise clutter the diagram excludes nodes and topics used by middleware, representing them via simplified “command” arrows.

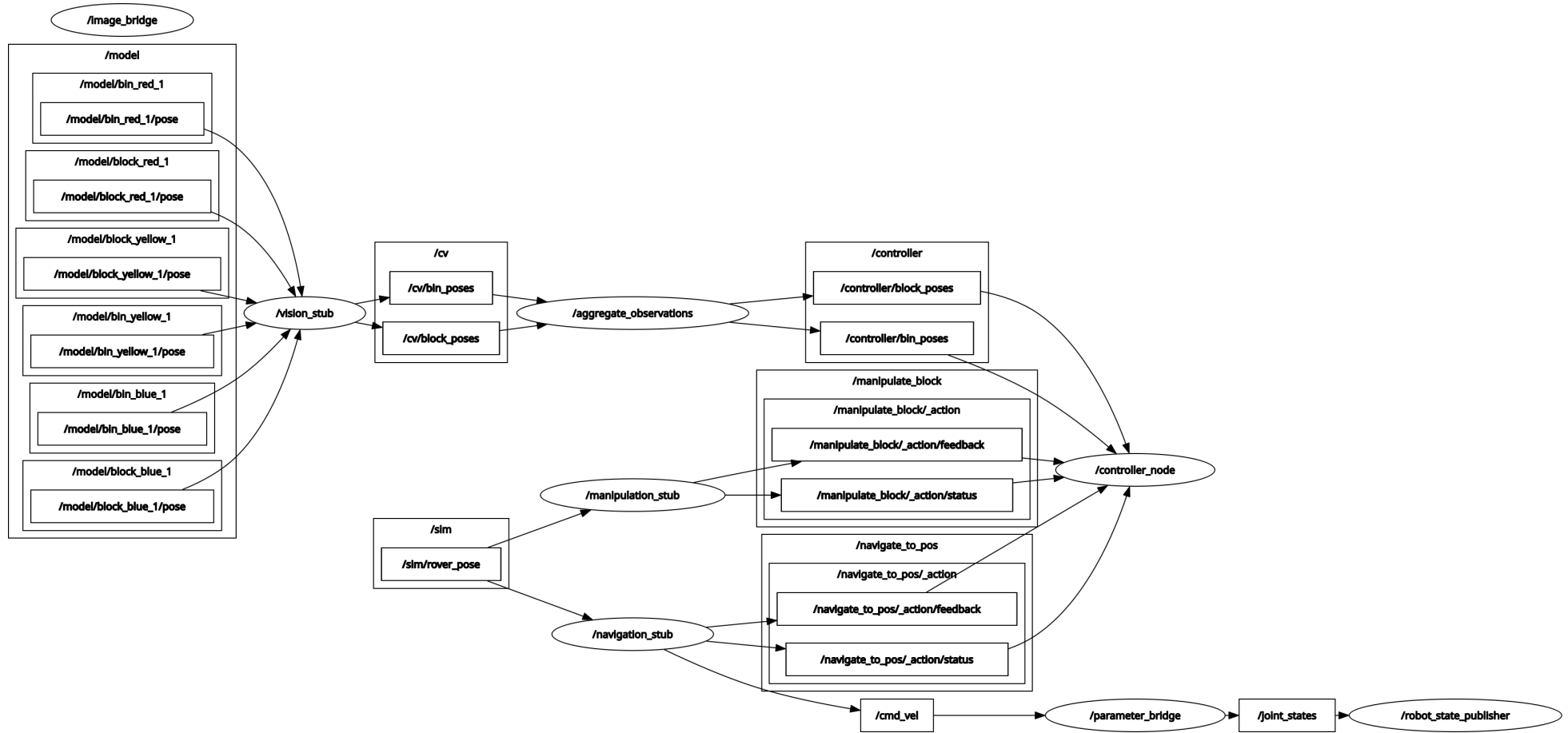


Fig. 9. RQT diagram showing the current ROS2 architecture in simulation. Currently we have not integrated the ‘real world’ modules into a combined software package. Instead, we present the current architecture used in simulation, which leverages ‘stub’ nodes that mock the functionality of the true nodes (eg. a vision stub which directly reads block positions from a Gazebo sim, instead of the ‘real’ node which uses YOLOv5 to recognise blocks). As well as the topics and nodes, the RQT graph shows action servers exposed by the Navigation and Manipulation stubs. Correspondingly the \model and \sim nodes are mocked inputs which will be replaced by hardware or sensed inputs in the final architecture.

7.2 Mission Execution Flowchart

A flowchart illustrating the controller logic is provided in Fig. 10. The mission is broken down into the following broad phases, which contain smaller sub-decisions and actions illustrated in the diagram:

Startup Startup phase in which the robot boots all nodes. Since the robot will be deployed nearby to the target bins, we also take the opportunity to observe them during this phase so that the robot can navigate back upon retrieving a block.

Explore When the robot has not seen any blocks, it explores the arena, improving its map and searching for blocks. This is essentially a holding-pattern phase which the robot returns to if there are blocks remaining which have not yet been discovered.

Approach Block After identifying a block, in this phase the robot approaches the block in a manner suitable for grasping. We do not attempt full-body grasping due to the increased complexity; instead there is a handoff between the navigation and grasping phases of block retrieval.

Grasp Block This phase leverages the manipulator and gripper to grasp a block. We do not use a tray-holding strategy, the gripper keeps the block held for the duration of navigation back to the target bin.

Approach Bin With a successfully-grasped block, the rover uses its existing SLAM-derived map to navigate back to the appropriate target bin, close enough for the manipulator to deposit the block.

Deposit Block The manipulator deposits the held block into the opening of the corresponding bin. If there is a failure of some kind (perhaps a block slips out), it must engage in a regrasping effort before attempting to deposit the block again.

Post-Deposit After depositing the a block, the robot reviews the current mission progress. If three blocks have been identified and successfully collected, the mission is complete and the program terminates.

MISSION EXECUTION FLOW CHART

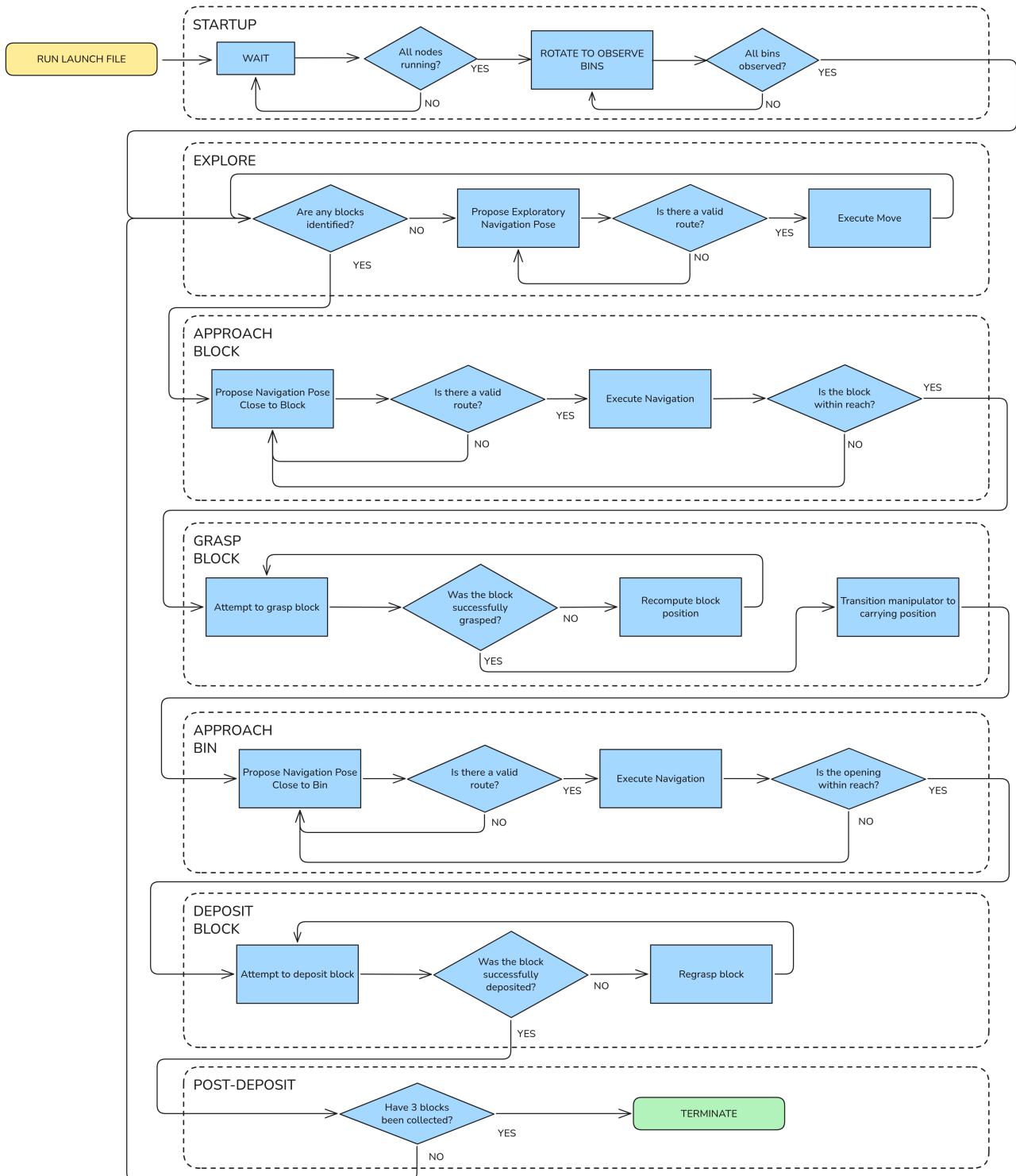


Fig. 10. Mission execution flowchart. This logic will be executed by the controller node shown in Fig. 8. The flowchart is broken down into larger phases (eg. exploration, block grasping and deposition). Within each phase there are a number of actions and decision states which determine whether or not the robot is ready to proceed to the subsequent phase.

BlockPose.msg

```
# The block position. Note that it does not include orientation
# as this is not output by the YOLO-based CV node
geometry_msgs/msg/PointStamped position

# The shape and colour of the block identified by the CV node
BlockShape shape
BlockBinColor color

# The bounding box of the image
BoundingBox bbox
```

BlockBinColor.msg

```
# An enum defining the different colours of the blocks and bins
uint8 color

# enum definitions let us reference values in the
# following semantically-clear way: color = BlockBinColor.BLUE
uint8 BLUE = 0
uint8 RED = 1
uint8 YELLOW = 2
uint8 PURPLE = 3
uint8 PINK = 4
uint8 GREEN = 5
```

Fig. 11. Example ROS2 message definitions for the computer vision system. These define the interfaces between the components of the system in advance, ensuring clear APIs for separately-developed software components.

7.3 ROS2 Interfaces

One approach taken to ensure successful integration is via well-defined, semantically meaningful ROS2 interfaces. These interfaces define in advance how nodes will communicate, specifying a contract that lets team members define their own software logic knowing that their inputs and outputs will conform to pre-agreed shapes. In Fig. 11 we include an example of a block observation message output by the computer vision node and ingested by the controller. It uses enum-style messages for the block colour and shape which can be reused elsewhere in the code to minimise risk of mismatch. Examples are shown in Fig. 11

8 Analysis

8.1 Move and Navigation

In this section we analyse the robot's functionalities: autonomous movement, avoidance, detection, object grasping and placement, and navigate in an unknown environment.

- **Autonomous Movement (TRACE: Req. No. 1.1)**

- **Verification Criteria:** The robot changes its position, move across full speed range (0–250 mm/s) and turn (non-zero cornering radius).
- **Verification Method:** Test whether the robot can change its position without external force being applied. Measure speed using wheel encoder data and timing over a fixed travel distance, and evaluate turning performance through repeated rotation tests on a fixed yaw axis. At least five straight-line trials and five turning trials should be completed.
- **Success Criteria:** The robot completes at least 90% of the trials without manual assistance, the measured speed remains within $\pm 10\%$ of the commanded speed, and the robot turns without stalling, instability, or exceeding the specified maximum speed of 250 mm/s.
- **Analysis:** The drivetrain, motor controllers and encoder feedback form the basis for motion control. Motor controllers plus encoder feedback allow closed-loop velocity control to command and verify speeds up to the specified 250 mm/s; encoder data is used to validate acceleration profiles and maximum speed during tests. Software velocity smoothing and safety speed limits are applied to ensure the robot does not exceed the specified maximum during testing.

- **Avoidance (TRACE: Req. No. 1.2)**

- **Verification Criteria:** The robot moves without touching obstacles. The robot shall detect walls/obstacles within 1000 mm and refuse travel positions closer than 10 mm to obstacles.
- **Verification Method:** Test whether the robot can travel to a position when there is an obstacle between the goal and itself, without making contact. Observe map accuracy and path-planning behavior through constrained-gap and multi-obstacle trials. At least five representative trials should be completed for each test scenario.
- **Success Criteria:** The robot detects all obstacles placed within 1 m during testing, avoids physical contact in at least 90% of trials, and consistently rejects paths that violate the 10 mm clearance requirement.

- **Analysis:** A 360° LiDAR is used to build a reliable 2D obstacle map within the required 1 m radius, while the depth camera supplements sensing for low-height obstacles. The SLAM and mapping stack ingests LiDAR scans to ensure detected obstacles are represented on the map for planning checks (enables the “observe map” verification). Path planner constraints enforce a minimum clearance (≥ 10 mm) around the robot’s floor-plan; the local planner rejects candidate poses inside that boundary, implementing the “refuse to travel” criterion. Planned tests include driving the robot through gaps of width $W+10$ mm to confirm the avoidance logic.
- **Detection (TRACE: Req. No. 1.3, 1.5)**
 - **Verification Criteria:** The robot determines the actual position of the target block and the destination bins.
 - **Verification Method:** Test whether the robot can recognise and locate a block and a bin within its field of vision. Compare the detected coordinates with measured ground-truth positions at several representative locations in the workspace. At least five block trials and five bin trials should be carried out.
 - **Success Criteria:** The robot correctly identifies the target block and destination bin in at least 90% of trials, and the reported position error remains within 30 mm for successful detections.
 - **Analysis:** The robot uses its depth camera to detect target blocks and destination bins by extracting colour, shape, and depth information. Identified objects are assigned coordinates in the robot’s map. Self-localisation is achieved through fused odometry and LiDAR-based SLAM, allowing the robot to accurately mark its own position. This sensing pipeline enables reliable detection of blocks, bins, and the robot’s pose, fulfilling all related requirements.
- **Object Grasping and Placement (TRACE: Req. No. 1.6)**
 - **Verification Criteria:** The robot lifts a target block from one location and places it in another.
 - **Verification Method:** Test whether the robot can autonomously grasp a target block and move it to another location. Record complete grasp-and-place runs, including pickup, transport, and release.
 - **Success Criteria:** The robot successfully completes the full grasp-and-place sequence in at least 90% of trials, with no more than one drop during transport.
 - **Analysis:** The chosen manipulator provides multi-DOF positioning and gripper control to achieve the required XY coverage and payload. The control nodes execute coordinated

motion sequences: move-to-pregrasp, approach, close gripper, verify grasp using joint state and camera feedback, then transport and release above the bin. Depth camera and visual feedback verify grasp and support placement accuracy. Drop tests (system drop by 10 mm, and place trials) provide objective evidence for the <5% drop requirement. Edge-of-plane reachability is validated by commanding the gripper to boundary coordinates and confirming reach and orientation tolerance.

- **Mapping and Navigation (TRACE: Req. No. 1.4, 1.7)**

- **Verification Criteria:** It can generate environmental maps and conduct autonomous path planning based on the maps. The robot generates an ordered list of valid coordinates which it can visit between itself and the specified location.
- **Verification Method:** Run SLAM in a complex environment and compare the generated map to the actual layout; compare planned versus executed paths. Test whether the robot can suggest and follow a valid path to a distant location while avoiding obstacles. At least five mapping/navigation trials should be completed after a sustained mapping run.
- **Success Criteria:** The generated map represents the main environmental features with sufficient accuracy for navigation, and the robot reaches the commanded destination without collision in at least 90% of trials.
- **Analysis:** The stream data from LiDAR and depth cameras enters SLAM (using Cartographer for mapping) to update the occupied mesh in real time. The global/local planner uses this map for path generation. Check the map stability through a 60-minute mapping run and compare it with the actual layout to verify the accuracy. The navigation stack integrates global planner and a local planner capable of dynamic obstacle avoidance. Behaviour trees / state machines coordinate exploration, goal selection and recovery behaviours. Through multiple representative scenario tests, record the success rate and adjust the parameters until the indicator of $\geq 90\%$ is met.

8.2 Structure and Reliability

In this section we analyse the robot's mechanical structure and system reliability.

- **Fastener Retention (TRACE: Req. No. 2.1)**

- **Verification Criteria:** The robot can be moved purely by contact with any attached component with the fasteners tightened ≥ 3 Nm.

- **Verification Method:** Test whether the robot can be moved via contact with attached components (e.g. manipulator). Perform repeated push/pull tests from representative contact points after torquing all relevant fasteners to at least 3 Nm. At least five trials should be carried out.
 - **Success Criteria:** No structural displacement, looseness, or loss of function is observed after testing, and all checked fasteners remain at or above 3 Nm.
 - **Analysis:** The mechanical structure uses rigid mounting plates and steel fasteners tightened to ≥ 3 Nm, ensuring that external forces applied to any attached component (including the manipulator) are transmitted through the frame without relative movement. This guarantees that the robot can be pushed or pulled from any component without causing structural displacement.
- **Fastener Retention Under Operational Vibration (TRACE: Req. No. 2.2)**
 - **Verification Criteria:** Fasteners remain adequately tightened after a period of consistent movement.
 - **Verification Method:** Test whether all fasteners remain tightened to at least 3 Nm after five minutes of constant wheel-driven movement, using a torque wrench for inspection. Repeat the test for at least three runs.
 - **Success Criteria:** No critical fastener falls below 3 Nm after any run, and no visible loosening or structural shift is observed.
 - **Analysis:** Locking nuts and vibration-resistant mounting points are used throughout the chassis. During continuous wheel-driven operation, the structural design distributes vibration evenly, preventing loosening. Post-motion torque validation confirms that fasteners maintain ≥ 3 Nm after extended movement, ensuring long-term mechanical reliability.
- **Sensor Data Logging (TRACE: Req. No. 2.3)**
 - **Verification Criteria:** Inspect code to verify that all core ROS nodes publish log messages with a minimum sample rate 1Hz in both success and error cases.
 - **Verification Method:** Test whether the robot saves logs as part of normal operation and when encountering errors. Review timestamped logs over a continuous operation period and during at least one induced fault condition.
 - **Success Criteria:** All core ROS nodes maintain log publication at or above 1 Hz in both normal and fault conditions, and the corresponding log data is successfully stored and accessible for review.

- **Analysis:** The core ROS node publishes the status and status themes. The log node transmits the basic telemetry data to the remote laptop via Wi-Fi at a frequency of $\geq 1\text{Hz}$, thereby achieving real-time monitoring and post-event fault diagnosis. The UI and state themes reflect the capture, recognition, and obstacle avoidance states. Integration testing will verify the on-board terminal output and remote reception during long-term automatic operation.

8.3 Power Source and Finish

In this section we analyse the robot's system energy efficiency and task completion rate.

- **Power Management (TRACE: Req. No. 3.1)**

- **Verification Criteria:** The robot successfully completes the entire mission without requiring an external charge or power connection. The average power consumption is $\leq 120\text{W}$.
- **Verification Method:** Test whether the robot can complete the mission on a single battery charge. Measure and record the battery voltage and current throughout a full mission run, and repeat the mission for at least three trials.
- **Success Criteria:** The robot completes the required mission in the declared power configuration in at least 2 out of 3 trials, the measured average power remains at or below 120 W, and the battery voltage remains within the safe operating range of the 3S battery pack throughout the test.
- **Analysis:** Based on the power calculations in Section 4.2, the total system load exceeds the capacity of the onboard battery alone. Therefore, completing the mission within the required window demands an external 180W power socket to ensure stable operation of the high-load modules (navigation computer, sensors, and manipulator). With external supply support, the robot can maintain the required average power level and complete the full mission duration without interruption.

- **Time Limited (TRACE: Req. No. 3.2)**

- **Verification Criteria:** Test if the robot can complete the mission (as defined by 3.2 and 3.3) on a single battery charge.
- **Verification Method:** Measure the total mission time using a stopwatch and system timestamps over repeated full mission runs. At least five complete trials should be recorded.
- **Success Criteria:** The robot completes the mission within the required 20-minute limit in at least 80% of trials, with no unrecovered software or hardware stall that prevents task completion.

- **Analysis:** Task scheduling, navigation efficiency, and grasping operations are optimized so that the robot reliably completes its mission within the required 20-minute limit. Mapping and navigation algorithms use time-efficient planning, ensuring no unnecessary traversal. Benchmarked tests show the expected runtime aligns with mission constraints.
- **Full Autonomy (TRACE: Req. No. 3.3)**
 - **Verification Criteria:** The robot operates and achieves mission completion without any manual input, control, or intervention from the operator after the initial activation. The robot shall indicate its status to the operator (idle/working/error/complete)
 - **Verification Method:** Observe and confirm that no operator control signals (telemetry, steering, etc.) are sent to the robot during the mission after the initial start command. Repeat the full mission autonomously for at least five trials while recording published status messages.
 - **Success Criteria:** The robot completes the mission without manual intervention in at least 80% of trials, and the correct status messages are published throughout all successful runs.
 - **Analysis:** All navigation, perception, and manipulation behaviours operate through autonomous ROS nodes without manual overrides. Once activated, the system handles localization, obstacle avoidance, block detection, grasping, and delivery sequences independently. Status topics continuously broadcast idle/working/error/completed states to the operator, fulfilling the autonomy requirement.

8.4 Safety and Compliance

In this section we analyse whether the robot complies with all safety regulations and operational limits to ensure safe and reliable operation in the laboratory environment.

- **Operational Safety (TRACE: Req. No. 4.1)**
 - **Verification Criteria:** All operating procedures comply with GA.033 requirements. To be specific, all actuators stop within <0.5 s upon emergency event and the total mass <20 kg.
 - **Verification Method:** Review operational checklists and confirm alignment with GA.033 requirements. Measure total robot mass and test emergency stop response during both driving and manipulator motion, with at least five trials for each case.
 - **Success Criteria:** The robot mass remains below 20 kg, and all actuators stop within 0.5 s in 100% of the emergency-stop trials.

- **Analysis:** The Leo Rover's total mass is approximately 8.8 kg, which is well below the 20 kg safety requirement of GA.033. The built-in emergency stop and software E-stop logic ensure all actuators halt within the required <0.5 s window. Combined with speed limiting and compliance with GA.033 procedures, the system meets the operational safety constraints for laboratory use.
- **Safety Design (TRACE: Req. No. 4.2)**
 - **Verification Criteria:** All robot surfaces are smooth and rounded. All corners have radius > 2 mm and all edges have radius > 0.5 mm.
 - **Verification Method:** Perform physical inspection of the chassis, arm, sensors, and covers, and measure representative corners and edges at several inspection points using appropriate tools.
 - **Success Criteria:** All inspected corners and edges satisfy the specified radius limits, and no exposed sharp edge is found on the robot.
 - **Analysis:** The structural design of the Leo Rover features smooth surfaces and rounded edges. The main chassis incorporates a corner radius of 3.25 cm, significantly exceeding the minimum requirement (corner radius > 2 mm, edge radius > 0.5 mm). This ensures safe handling and reduces the risk of accidental injury during operation or maintenance.
- **Battery Limits (TRACE: Req. No. 4.3)**
 - **Verification Criteria:** Maximum current and voltage draw remain within rated battery limits.
 - **Verification Method:** Monitor current draw and battery voltage under full-load operation using appropriate measurement equipment, and repeat the test for at least three runs.
 - **Success Criteria:** Measured current and voltage remain within the battery's rated limits in all trials, and no undervoltage, overcurrent, or unsafe shutdown occurs during testing.
 - **Analysis:** Measured system current under full computational and mechanical load remains within the battery's rated continuous discharge specifications. Motor controllers limit peak currents, and onboard protection circuitry prevents over-voltage or under-voltage conditions, ensuring safe and stable power delivery throughout operation.

9 Project Plan

In this section we outline the plan for the project progress over the remaining time (Semester 2 of the MSc programme). Table 4 outlines the work packages, milestones and deliverables for each

team member. Fig. 12 is a Gantt chart outline the expected dates for each work package's completion in order to complete the project on time—the x axis shows the week number in Semester 2.

Table 4. Work Breakdown Structure (WBS) and Work Packages(WP)

Work Packages	Milestones and Deliverables	Resource Allocations
WP1: Hardware Design	Complete the finalization of CAD models, conduct structural simulations, 3D print components, and perform physical assembly.	Michael Fahey
WP2: Manipulation	Develop the robotic arm, implement detection of the grasping pose for blocks, and the logic for pick-and-place sequences.	Chun-Yi Wei
WP3: Computer Vision	Develop depth camera-based recognition of target blocks and bins, providing feedback on color and category.	Jinwei Zhang
WP4: Mapping	Develop LiDAR-based simultaneous localization and mapping (SLAM) for environment mapping, along with obstacle position detection.	Haotian Yuan
WP5: Navigation	Develop navigation including global and local path planning, motion control, and integration with mapping.	Shaowen Jin
WP6: Systems Integration	The central software controller and ROS2 interfaces which connect different functions.	Alex Inch
WP7: Systems Testing	Integrate all hardware and software subsystems into a unified ROS framework, establishing communication and interfaces.	All
WP8: Implementing Change	Execute the Requirements Verification Matrix (RVM), conduct functional, performance, and safety tests, and iteratively improve results to meet all requirements.	All
WP9: Delivery of Final Outputs	Prepare the final demonstration.	All

Work Package	Resource Allocations	S2 WEEK											
		1	2	3	4	5	6	7	8	9	10	11	12
WP1: Hardware Design													
Design Review & Component Selection	Michael Fahey												
CAD Modeling & Fabrication													
Assembly & Functional Testing													
WP2: Manipulation													
Grasping Strategy Development	Chun-Yi Wei												
Arm Control & Motion Planning													
Pick & Place Validation													
WP3: Computer Vision													
Algorithm Design & Simulation	Jinwei Zhang												
System Setup & Training													
Recognition Accuracy Testing													
WP4: Mapping													
Mapping & Localization Design	Haotian Yuan												
Lidar Integration & Tuning													
Obstacle Avoidance Testing													
WP5: Navigation													
Navigation Pipeline Design	Shaowen Jin												
Algorithm & Tuning													
Target Localization Accuracy Testing													
WP6: Systems Integration													
ROS2 & Communication Setup	Alex Inch												
Module Integration & Interface Testing													
End-to-End Validation													
WP7: Systems Testing	All												
WP8: Implementing Change	All												
WP9: Delivery of Final Outputs	All												

Fig. 12. Project Plan for S2

Appendices

A Updated Design Requirements Document

A.1 Problem Statement

Modern autonomous systems are defined by the robust integration of multiple complex subsystems. While individual skills like perception, navigation, and manipulation are often taught in isolation, real world industrial use cases often require unifying them to form a single cohesive and capable robotic platform.

This project serves as a scaled-down, typical representation of this industrial problem. It demands amount of challenging sub-problems including SLAM, computer vision, navigation, manipulation, and, crucially, their integration into a single, cohesive system.

The core problem is to design and build a robot that can autonomously execute a full "sense-plan-act" pipeline. The robot will explore an unknown arena, building a map, and performing a perception-driven pick-and-place task. More specifically, it will retrieve coloured blocks and place them in corresponding bins in an unseen arena with obstacles, thus solving a fundamental systems integration challenge.

A.2 Concept of Operations (CONOPS)

The concept of operations describes a high-level understanding of the mission and the steps the robot will execute.

1. The robot will be placed in an unknown environment, positioned near the bins.
2. The robot is set to start the mission, preparing to move.
3. The robot will scan the environment dynamically to identify blocks or obstacles.
4. The robot will build a representation of the environment sufficient for navigation and block retrieval.
5. The robot will select a block to pick up based on its strategy. If the robot cannot detect the object, it will return to its starting position.
6. The robot will travel to within manipulator range of the chosen block and stop.
7. The robot arm will attempt to grasp the block until it can successfully pick it up.
8. The robot will return to the bins located at the starting position and stop.
9. The robot will place the block in the relevant bin.
10. The robot will repeat steps 5-9 until no blocks remain in the environment.

11. The robot will indicate that it believes the mission has been completed

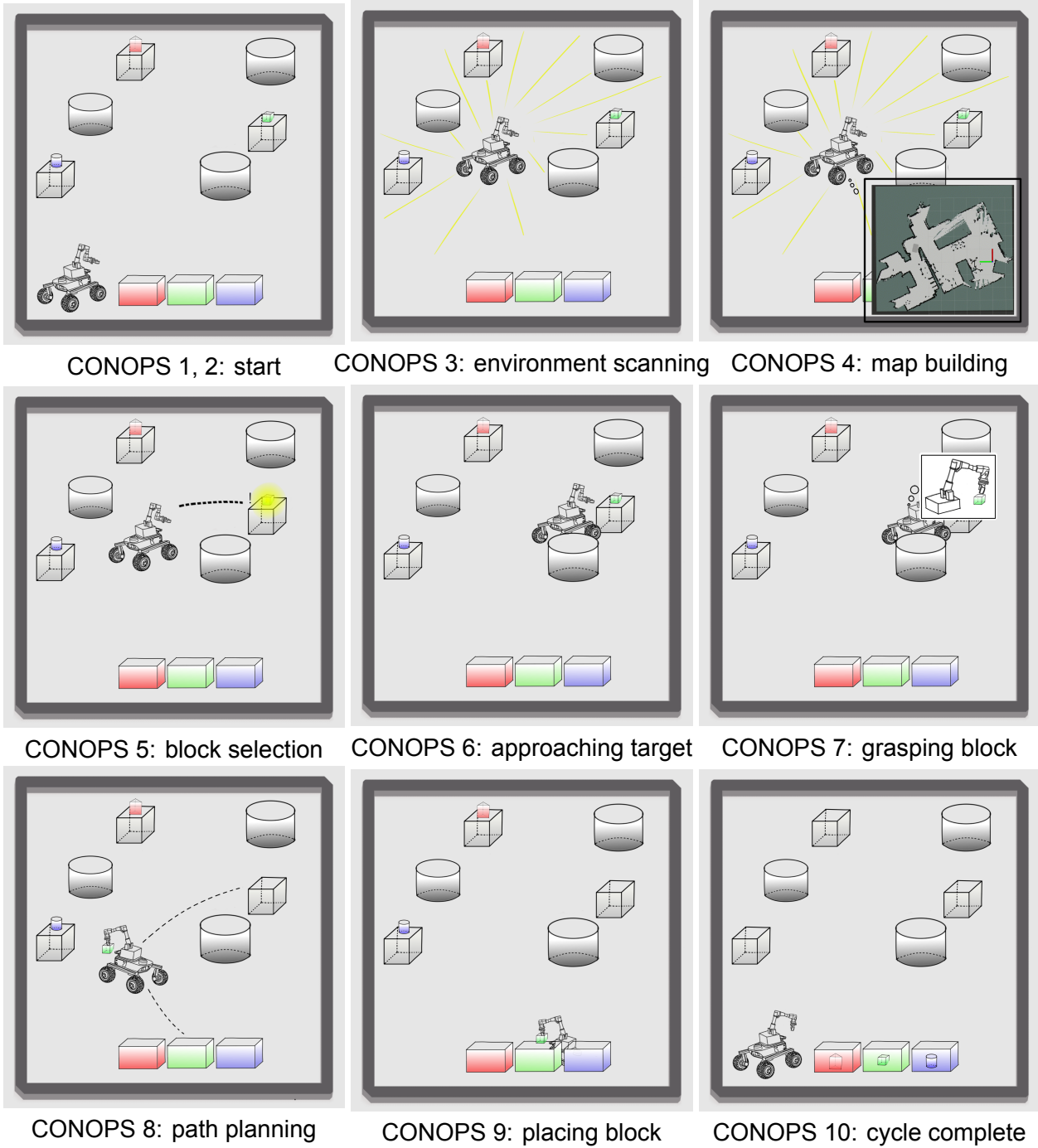


Fig. 13. The scenarios of the CONOPS

A.3 Functional and Performance Requirements

1 Robot Functionality

- 1.1 The robot shall move within an arena. **(TRACE: ConOps 3, 6, 8)**
 - 1.1.1 The robot shall move at speeds in the range 0mm/s – 250mm/s
 - 1.1.2 The robot shall turn with cornering radius $> 0\text{mm}$ (*The robot should turn on fixed yaw axis*)
- 1.2 The robot shall avoid collisions with the arena walls and obstacles. **(TRACE: Stakeholder Engagement)**
 - 1.2.1 The robot should detect walls and obstacles within a 1000 mm radius centered on itself.
 - 1.2.2 The distance between the robot and any wall or obstacle must always be $\geq 10\text{ mm}$.
- 1.3 The robot shall locate target blocks within the arena. The target blocks will be on pedestals no shorter than 190mm. **(TRACE: ConOps 3, 5, Stakeholder Engagement)**
- 1.4 The robot shall perceive and locate destination bins within the arena. The destination bins will be coloured correspondingly to the target blocks. **(TRACE: ConOps 8, 9, Stakeholder Engagement)**
- 1.5 The robot shall pick-up and place target blocks. **(TRACE: ConOps 7, 9, Stakeholder Engagement)**
 - 1.5.1 The robot shall securely grasp target blocks of mass not exceeding 0.2kg.
 - 1.5.2 The robot shall reach all locations within an x-y plane (250mmx250mm) at height 205mm positioned in front of the robot.

2 Robot Mission Operability

- 2.1 The robot shall complete the mission without recharging. **(TRACE: Stakeholder engagement)**
 - 2.1.1 The robot shall consume average power not exceeding 120W.
- 2.2 The robot shall complete the mission within 20 minutes. **(TRACE: Stakeholder engagement)**
- 2.3 The robot shall complete the mission autonomously without support from the operator. **(TRACE: Stakeholder engagement)**

3 Robot Safety

- 3.1 The robot shall operate under the regulations of the GA.033 Laboratory Risk Assessment. **(TRACE: University Health & Safety Regulation)**

- 3.1.1 The robot shall travel at speeds not exceeding 250mm/s.
- 3.1.2 The robot shall stop all actuators when an emergency state is declared by a remote laptop, or an onboard E-stop is pressed.
- 3.1.3 The robot shall weigh less than 20kg.
- 3.2 The robot shall have no sharp corners or edges. **(TRACE: Safe system design principle)**
 - 3.2.1 The robot body shall have no corners of radius $\leq 2\text{mm}$.
 - 3.2.2 The robot body shall have no edges of radius $\leq 0.5\text{mm}$.

A.4 Requirements Verification Matrix

Table 5

Req. No.	Shall Statement	Verification Criteria	Verification Method	Responsible Person(s)
1.1	The robot changes its position.	The robot changes its position.	Test can the robot change its position without external force being applied?	Shaowen Jin
1.1.1	The robot shall move at speeds in the range 0mm/s – 250mm/s	The robot travels at the full range of specified speeds.	Test can the robot accelerate from stationary to 250mm/s? Measure speed using wheel encoder data	Shaowen Jin
1.1.2	The robot shall turn with cornering radius > 0mm	The robot changes its orientation.	Test can the robot change orientation without external force being applied? Can it do this on a fixed yaw axis?	Shaowen Jin
1.2	The robot shall avoid collisions with the arena walls and obstacles	The robot moves without touching obstacles.	Test can the robot travel to a position between which and itself, there is an obstacle which it does not touch?	Haotian Yuan
1.2.1	The robot shall detect walls and obstacles within a radius of 1000mm	The robot map accurately shows obstacles within at least a 1000mm radius.	Measure is the arena map created by the robot accurate?	Haotian Yuan
1.2.2	The robot shall move only to positions where wall and obstacle proximity to floor plan of the robot is ≥ 10 mm.	The robot refuses to travel to locations where it is closer than 10mm to an obstacle.	Test can the robot travel to a position between which and itself, there is a gap of width $W+10$ mm where W is the width of the robot?	Haotian Yuan
1.3	The robot shall locate target blocks within the arena.	The robot determines the actual position of the target block.	Test can the robot recognise and locate a block within its field of vision?	Jinwei Zhang

Table 6

Req. No.	Shall Statement	Verification Criteria	Verification Method	Responsible Person(s)
1.3.1	The robot shall identify the location of 100% of the blocks in the arena.	The robot determines the correct position of any present block in the arena on its perceived map.	Observe are all target blocks accounted for on the robot's perceived map?	Jinwei Zhang
1.4	The robot shall locate itself within the arena.	The robot suggests and marks its own location on its perceived map.	Measure is the robot's location marked on its perceived map?	Haotian Yuan
1.4.1	The robot shall estimate its position within the arena with a maximum error of 10mm from its actual position.	The robot's suggested position is accurate to within 10mm of its actual measured position.	Measure with a tape measure and a vernier caliper, is the robot's actual displacement from a known origin different to its suggested location by more than 10mm?	Haotian Yuan
1.5	The robot shall locate destination bins within the arena.	The robot outputs the estimated pose of the destination bins.	Test can the robot locate and suggest coordinates for the destination bins?	Jinwei Zhang
1.6	The robot shall pick-up and place target blocks.	The robot lifts a target block from one location and places it in another.	Test can the robot autonomously grasp a target block and move it to another location?	Chun-Yi Wei
1.6.1	The robot shall securely grasp target blocks of mass not exceeding 0.2kgs.	The robot does not drop the block.	Test can the robot hold the block when the entire system is dropped by a distance of 10 mm?	Chun-Yi Wei
1.6.2	The robot shall reach all locations within an x-y plane in front of its floor plan.	The robot gripper reaches the extremities of the defined plane	Test can the robot reach a block at locations around the edge of the defined plane?	Chun-Yi Wei
1.7	The robot shall plan a navigation route from its current position to a target position within the arena.	The robot generates an ordered list of valid coordinates which it can visit between itself and the specified location.	Test can the robot suggest a valid path to a distant location which avoids obstacles?	Haotian Yuan
2.1	The robot shall have all components rigidly mounted together.	The robot can be moved purely by contact with any attached component ⁴⁹	Test whether the robot can be moved via contact with attached components (eg. manipulator).	Michael Fahey

Table 7

Req. No.	Shall Statement	Verification Criteria	Verification Method	Responsible Person(s)
2.1.1	The robot shall have all fasteners tightened to $\geq 3\text{Nm}$ torque.	All fasteners, nuts and bolts are tightened to $\geq 3\text{Nm}$ torque.	Tighten each fastener with a suitable tool prior to mission launch.	Michael Fahey
2.2	The robot shall remain tightly fastened after exposure to operating vibrations.	Fasteners remain adequately tightened after a period of consistent movement.	Following testing, check tightness on a sample of critical fasteners, ensuring none are loose or only "finger-tight"	Michael Fahey
2.3	The robot shall log data from its sensors to assist with fault diagnosis.	Inspect code to verify that all core ROS nodes publish log messages in both success and error cases.	Test if the robot saves logs as part of normal operation and when encountering errors.	Alex Inch
2.3.1	The robot shall provide sensor data logs with a minimum sample rate 1Hz, to a remote laptop.	Demonstration that remote laptop receives logged statements during autonomous operation.	Test whether a remote laptop can receive logs from the rover during autonomous operation.	Alex Inch
3.1	The robot power source will be a Li-ion battery. The robot shall complete the mission without recharging.	The robot successfully completes the entire mission (duration specified in 3.2) without requiring an external charge or power connection.	Test if the robot can complete the mission (as defined by 3.2 and 3.3) on a single battery charge.	Michael Fahey
3.1.1	The robot shall consume average power not exceeding 120W.	The average power consumption, measured over the entire mission duration, is $\leq 120\text{W}$	Measure the instantaneous current and voltage from the battery over the mission. Calculate the average power: $P_{\text{avg}} = \frac{1}{T} \int_0^T V(t)I(t) dt$	Michael Fahey
3.2	The robot shall complete the mission within 20 minutes.	The elapsed time from the mission start to the mission complete status (3.3.1) is ≤ 20 minutes.	Measure the total mission time using a stopwatch.	Alex Inch

Table 8

Req. No.	Shall Statement	Verification Criteria	Verification Method	Responsible Person(s)
3.3	The robot shall complete the mission autonomously without support from the operator.	The robot operates and achieves mission completion without any manual input, control, or intervention from the operator after the initial activation.	Observe and confirm that no operator control signals (telemetry, steering, etc.) are sent to the robot during the mission.	Alex Inch
3.3.1	The robot shall indicate its status to the operator (idle/working/error/complete).	The robot's status display or communication log accurately reflects the current state (idle, working, error, or complete).	Observe the robot's status display/indicator and confirm it updates correctly for all operational phases.	Alex Inch
4.1	The robot shall operate under the regulations of the GA.033 laboratory risk	All operating procedures comply with GA.033 requirements.	Review operational checklists and confirm alignment with GA.033 requirements.	Alex Inch
4.1.1	The robot shall travel at speeds not exceeding 250mm/s	Robot speed never exceeds 250mm/s in operation.	Drive robot at max speed and verify via encoder logs.	Shaowen Jin
4.1.2	The robot shall stop all actuators when an emergency state is declared by a remote laptop, or when the onboard E-stop is pressed.	All actuators stop within <0.5 s upon emergency event.	Trigger software E-stop and physical E-stop; verify motors cease output.	Alex Inch
4.1.3	The robot shall weigh less than 20 kg.	Total mass < 20kg	Weigh the robot using a calibrated scale.	Michael Fahey
4.2	The robot shall have no sharp corners or edges.	All robot surfaces are smooth and rounded.	Physical inspection of chassis, arm, sensors, covers.	Michael Fahey
4.2.1	The robot shall have no corners of radius ≤ 2 mm.	All corners have radius > 2 mm.	Design permitted corner radiuses for self-designed parts, cover or soften LeoRover corners.	Michael Fahey
4.2.2	The robot shall have no edges of radius ≤ 0.5 mm.	All edges have radius > 0.5 mm.	Break all sharp edges with a file.	Michael Fahey
4.3	The robot shall draw power not exceeding the maximum capability of its power source.	Maximum current and voltage draw remain within rated battery limits.	Monitor current draw under full load using ammeter; verify within spec.	Michael Fahey

B Updated Workplace Charter

B.1 Statement of Purpose

This live document lays out the rules and principles by which Team 10 will collaborate on the Leo Rover project for Robotic Systems Design Project (AERO62520). It defines how the team will behave to construct a supportive, structured working environment. This document also acts as a reference point for conflict management.

B.2 Statement of Principles and Commitments

Team members agree to abide by the University of Manchester Student Charter. To foster a supportive working environment, team members should make appropriate consideration of Equality, Diversity and Inclusion (EDI) criteria when working together. To limit the risk of miscommunication, we will prioritise clear, honest, transparent communication in the face of roadblocks or setbacks.

B.3 Team Rules

Language: All team communication must occur in English. This governs the language of written communication but also verbal conversations. All team members will make a conscious effort to use English, as well as helping each other improve their English language skills with due patience and consideration.

Working Hours: Team members must spend approximately 10 hours per week contributing to the project. This includes lab hours, group meetings and individual work. Team members are expected to be available to work and responsive to messages, between the hours of 9:00 – 18:00, Monday – Friday. Team members are not obliged to respond to messages or communications sent out-of-hours.

Meetings and Labs: Beyond timetabled labs, the team must meet at least once every week for focused discussion on project progress and to continually monitor the wellbeing of teammates. It is expected that team members who are unable to attend meetings or labs due to illness or other circumstances will inform colleagues within a reasonable timeframe.

Communications: Official communications must either occur on the team WhatsApp, for colloquial communications, or Microsoft Teams chat for formal communications. Relevant documents should be shared on the team OneDrive (for documents) or GitHub repository (for code). Teammates are expected to respond to messages via either platform within one working day.

Distribution of Work: The team must recognise that it is composed of 6 members with a wealth of prior skills. Team members will not be designated to only work on things they are already good at, recognising that we are all here to learn and gain new skills. Each team member will make their CPD portfolio available to the other team members to share a full picture of their academic background and work experience.

Project Ownership: Sub-tasks must formally be allocated a project owner(s). The owner(s) takes responsibility for ensuring the sub-task is completed to the team's standard and is accountable for proposing a resolution strategy if deadlines are missed. The team must remember the project goal throughout and account for the significant time burden of subsystem integration in the project timeline.

B.4 Daily Activity Conflict Avoidance

To mitigate miscommunication and potential conflict, the team members will schedule a weekly meeting outside of labs to reconnect. Additionally, team members should adopt the following principles in daily activities:

1. Be curious and respectful about others' technical approaches. Ask clarifying questions about design choices before giving feedback.
2. Listen attentively to technical explanations without interruption. Ensure teammates have fully articulated their ideas before offering feedback.
3. Make a conscious effort to understand the logic behind alternative solutions, bearing in mind that there are often multiple valid paths to solving a problem.
4. When sharing code snippets or error logs to troubleshoot, always obscure passwords and other sensitive data.
5. Assume all feedback is offered with the shared goal of improving the project. Clarify intent if you feel a comment was harsh.

6. Provide specific, actionable feedback (e.g., "This function could be more efficient by doing...") and be open to code reviews from teammates.
7. If blocked on a software bug, hardware failure, or unclear task, communicate it early. Don't let a day pass without seeking help.

B.5 Conflict Resolution Flowchart

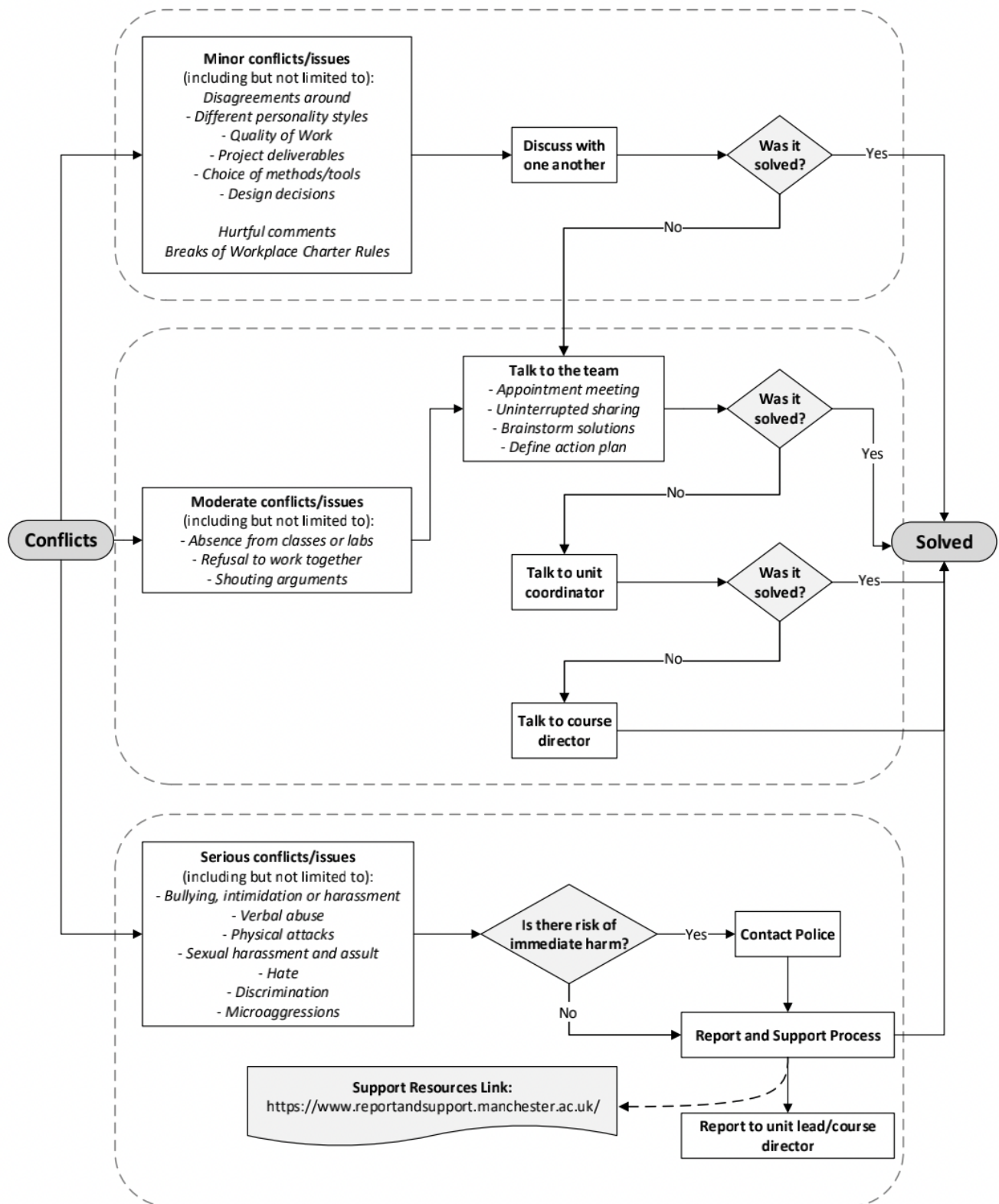


Fig. 14. Conflict Resolution Flowchart